GIẢI PHẪU MỘT
AGENTIC OPERATING SYSTEM

18 Architectural Patterns từ 513.000 dòng code Claude Code

Phân tích, đúc kết, và ứng dụng

* * *

Lâm Nguyễn

AI Officer, Real-Time Robotics

Tháng 4, 2026

Lời nói đầu

Cuốn sách này bắt đầu từ một sự cố tình cờ: ngày 31 tháng 3 năm 2026, source code đầy đủ của Claude Code bị lộ qua một file source map trong npm package chính thức. 513.000 dòng TypeScript. Không phải prototype hay demo. Đây là production code đang phục vụ hàng triệu developer.

Tôi đọc code đó không phải vì tò mò. Tôi đọc vì đang xây một thứ tương tự: Vibecode Kit, phương pháp luận AI-assisted development mà tôi phát triển để xây dựng phần mềm enterprise bằng AI tại Real-Time Robotics. Tôi muốn biết: production system thật giải những bài toán mà tôi đang vật lộn thế nào?

Câu trả lời là 18 architectural patterns. Mỗi pattern được verify bằng cách trace đến file, đến dòng, đến code comment cụ thể. Mỗi pattern là kiến thức phổ quát — async generator, concurrency partitioning, escalating recovery — không phải intellectual property. Chúng áp dụng được cho bất kỳ agentic system nào, không chỉ cho coding tool.

Cuốn sách này không reproduce code. Nó extract wisdom. Tôi hy vọng bạn tìm thấy những câu trả lời tương tự những gì tôi tìm thấy.

Lâm Nguyễn

Sài Gòn, tháng 4 năm 2026

Mục lục

PHẦN I

NỀN TẢNG

Tại sao cần "hệ điều hành" cho AI agents

* * *

CHƯƠNG 1

Từ chatbot đến agentic system

Giải phẫu một Agentic Operating System
Lâm Nguyễn

> *"Sự khác biệt giữa một tool và một agent không nằm ở mô hình ngôn ngữ bên trong. Nó nằm ở vòng lặp bên ngoài."*

1.1 Một cuộc gọi API không phải là một agent

Vào năm 2023, khi ChatGPT bùng nổ, hầu hết mọi người tiếp cận AI theo cùng một mô hình: gửi prompt, nhận response, hiển thị cho user. Mô hình này đơn giản đến mức một developer junior có thể implement trong một buổi chiều. Nó cũng đủ mạnh để tạo ra hàng nghìn sản phẩm thương mại.

Nhưng mô hình này có một giới hạn cơ bản: nó là one-shot. Mỗi lần gọi API là một giao dịch độc lập. AI trả lời xong thì quên. Nếu câu trả lời sai, user phải tự nhận ra và hỏi lại. Nếu task phức tạp cần nhiều bước, user phải chia nhỏ thủ công và gọi từng bước. AI không biết bước trước trả lời gì cho bước sau, trừ khi user copy-paste.

Đây không phải vấn đề của mô hình ngôn ngữ. GPT-4 hay Claude đều đủ thông minh để phân tích một codebase 50.000 dòng nếu được cho đủ context. Vấn đề nằm ở lớp điều phối bên ngoài mô hình. Khi bạn gọi API một lần và hiển thị kết quả, bạn đang dùng AI như một tool. Khi bạn để AI tự lặp, tự gọi tool, tự kiểm tra kết quả cho đến khi xong, bạn đang có một agent.

Sự khác biệt này nghe nhỏ nhưng hệ quả rất lớn. **Một tool cần con người điều khiển mỗi bước. Một agent cần con người đặt mục tiêu rồi giám sát kết quả.** Từ "giám sát" mới là keyword. Không phải "thay thế" con người, mà thay đổi vai trò con người: từ người lái xe thành người giám sát xe tự lái.

1.2 Năm mức trưởng thành: từ Manual đến Agent

Để hiểu vị trí của agentic system trong bức tranh lớn, ta cần một thang đo. Mô hình 5 mức AI maturity sau đây phân loại cách tổ chức sử dụng AI, từ thô sơ nhất đến tự trị nhất:

Mức	Tên	Mô tả	Ví dụ
1	**Manual**	Con người làm mọi thứ. AI không tham gia.	Viết code bằng tay, deploy thủ công
2	**Tool**	AI là công cụ gọi một lần. Con người điều khiển từng bước.	ChatGPT chat, GitHub Copilot autocomplete
3	**Assistant**	AI giữ context qua nhiều bước. Con người vẫn ra lệnh từng bước nhưng AI nhớ bước trước.	Claude Chat với memory, Cursor chat
4	**Copilot**	AI đề xuất hành động. Con người approve hoặc reject. AI thực thi sau khi được duyệt.	Claude Code (chế độ hỏi permission), Vibecode Chủ thầu/Thợ
5	**Agent**	AI tự lặp, tự gọi tool, tự kiểm tra. Con người đặt mục tiêu và giám sát kết quả.	Claude Code (auto mode), multi-agent swarms

Hầu hết sản phẩm AI hiện tại nằm ở mức 2 hoặc 3. Một số ít đạt mức 4. Rất ít đạt mức 5 ở production. Claude Code là một trong số ít sản phẩm đạt mức 5 với hàng triệu developer sử dụng hàng ngày. Đó là lý do kiến trúc của nó đáng nghiên cứu.

Quan trọng hơn, mỗi mức không thay thế mức trước. Mức 5 không có nghĩa "loại bỏ con người." Nó có nghĩa con người chuyển từ vai trò "người thực thi" sang "người giám sát." Trong Claude Code, ngay cả ở auto mode, hệ thống vẫn có permission gates, cost thresholds, và abort mechanisms cho phép con người can thiệp bất kỳ lúc nào.

1.3 Bài toán mà framework không giải được

Khi ý tưởng về AI agents lan rộng, hàng loạt framework xuất hiện: LangChain, AutoGPT, CrewAI, Autogen. Mỗi framework giải quyết một phần của bài toán. Nhưng

chúng đều chia sẻ cùng một giới hạn: chúng là framework, không phải operating system.

Sự khác biệt giữa framework và operating system ở đây là gì?

Framework cho bạn building blocks: cách gọi LLM, cách define tool, cách chain steps. Bạn assemble chúng thành ứng dụng. Khi ứng dụng chạy, framework không can thiệp. Nếu agent hết context window, bạn tự xử lý. Nếu hai agents sửa cùng file, bạn tự giải conflict. Nếu agent chạy lệnh nguy hiểm, bạn tự viết guard.

Operating system quản lý tài nguyên, phân quyền, và điều phối. Nó tự compact context khi gần hết. Nó tự phân loại tool calls thành concurrent hay exclusive. Nó tự classify commands thành safe hay dangerous. Nó cung cấp permission system, memory system, recovery system. Bạn viết agent logic; OS lo phần còn lại.

Đây chính xác là những gì Claude Code đã xây. Khi phân tích 513.000 dòng source code, ta không tìm thấy một "agent framework." Ta tìm thấy một hệ điều hành cho AI agents, với 6 pipeline lồng nhau, 43 tools, 26 lifecycle hooks, 5 lớp context defense, và permission classifier phân loại hàng nghìn bash commands.

1.4 Lịch sử ngắn: từ ReAct đến sản phẩm production

Con đường từ ý tưởng "AI có thể tự hành động" đến sản phẩm production đi qua 4 giai đoạn:

Giai đoạn 1: ReAct (2022)

Paper "ReAct: Synergizing Reasoning and Acting" của Yao et al. đề xuất ý tưởng cốt lõi: thay vì chỉ reasoning (chain-of-thought) hoặc chỉ acting (tool use), kết hợp cả hai trong một vòng lặp. AI suy nghĩ, hành động, quan sát kết quả, rồi suy nghĩ tiếp. Vòng lặp Thought-Action-Observation trở thành nền tảng cho mọi agentic system sau này.

Giai đoạn 2: Framework explosion (2023)

LangChain, LlamaIndex, Haystack, và hàng chục framework khác implement ReAct loop. AutoGPT gây sốt với demo tự động hoàn thành task phức tạp. Nhưng production deployment thì thất bại. Lý do: framework cho bạn vòng lặp, nhưng không cho bạn cách dừng vòng lặp, cách xử lý khi vòng lặp sai, cách quản lý context khi vòng lặp chạy quá lâu.

Giai đoạn 3: Wrapper apps và bài học cay đắng (2023-2024)

Hàng nghìn startup xây "AI agents" bằng cách wrap API calls trong giao diện đẹp. Khi model provider cải thiện capability, giá trị của wrapper giảm dần. Bài học được đúc kết: wrapper app luôn thua model vendor ở generic capability. Chỉ domain-specific orchestration mới có defensible value. Đây là insight mà Lâm đã viết trong bài "Cẩn thận với Agent nhà làm" trên cộng đồng AI Việt Nam.

Giai đoạn 4: Production agentic systems (2025-2026)

Một số sản phẩm vượt qua giai đoạn demo để trở thành production system: Claude Code, GitHub Copilot Workspace, Cursor Agent, Devin. Điểm chung: chúng không phải "wrapper." Chúng đầu tư nặng vào orchestration layer, nghĩa là vào context management, permission, concurrency, recovery, và extension mechanisms. Claude Code đầu tư 513.000 dòng code cho lớp này. Đó là quy mô của một hệ điều hành, không phải của một wrapper.

1.5 Tại sao cuốn sách này tồn tại

Vào ngày 31 tháng 3 năm 2026, source code đầy đủ của Claude Code bị lộ qua một file source map trong npm package chính thức. Đây không phải code thử nghiệm hay prototype. Đây là production code đang phục vụ hàng triệu developer, được đội ngũ engineering hàng đầu thế giới về AI xây dựng qua nhiều năm.

Sự kiện này tạo ra một cơ hội chưa từng có: lần đầu tiên, cộng đồng có thể nhìn vào bên trong một production agentic operating system ở quy mô 513.000 dòng code. Không phải đọc blog post mô tả kiến trúc từ xa. Không phải suy đoán từ API documentation. Mà đọc từng dòng code, từng comment, từng design decision.

Cuốn sách này là kết quả của việc đọc đó. Nó không reproduce code. Nó extract patterns: 18 architectural patterns đã được verify bằng cách trace từng pattern đến file, đến dòng, đến code comment cụ thể. Mỗi pattern là kiến thức phổ quát, không phải intellectual property. Chúng áp dụng được cho bất kỳ agentic system nào, không chỉ cho coding tool.

> *Một bản thiết kế 513.000 dòng code không tự nói. Nó cần người đọc biết đặt câu hỏi đúng. Cuốn sách này là tập hợp những câu hỏi đúng và câu trả lời tìm được.*

1.6 Ai nên đọc cuốn sách này

Cuốn sách này viết cho ba nhóm người:

Nhóm 1: AI Engineer. Bạn đang xây hoặc sẽ xây agentic system. Bạn cần biết production system thật giải quyết concurrency, context management, permission, và recovery thế nào. Cuốn sách cho bạn 18 patterns với replication guides.

Nhóm 2: Engineering Leader. Bạn cần quyết định kiến trúc cho AI platform của tổ chức. Bạn cần hiểu trade-offs giữa framework vs operating system approach, giữa single-agent vs multi-agent, giữa autonomy vs human-in-the-loop. Cuốn sách cho bạn 5 applied blueprints.

Nhóm 3: AI-assisted developer. Bạn dùng Claude Code, Cursor, hoặc tool tương tự hàng ngày. Bạn muốn hiểu vì sao tool đôi khi "nghĩ" rất lâu, vì sao nó hỏi permission, vì sao context bị compact. Hiểu kiến trúc giúp bạn dùng tool hiệu quả hơn.

1.7 Cấu trúc cuốn sách

Cuốn sách chia thành 4 phần:

Phần I (Chương 1-2): Nền tảng. Bạn đang ở đây. Chương này xác lập bối cảnh. Chương 2 vẽ bức tranh tổng thể: 6 pipeline lồng nhau và cách chúng phối hợp.

Phần II (Chương 3-5): Engine. Deep dive vào query loop (trái tim), tool orchestration (tay chân), và permission pipeline (hệ miễn dịch). Ba chương này giải thích cách hệ thống thực sự chạy.

Phần III (Chương 6-8): Swarm. Multi-agent coordination, background tasks, và context defense. Ba chương này giải thích cách hệ thống scale từ một agent lên nhiều agent mà không mất kiểm soát.

Phần IV (Chương 9-16): Ecosystem. Skills, Plugins, UI engine, infrastructure, phương pháp luận Vibecode, blueprints ứng dụng, triết lý human-in-the-loop, và tương lai. Phần này kết nối kiến trúc với thực tiễn.

* * *

1.8 Quy ước và thuật ngữ

Cuốn sách sử dụng một số thuật ngữ xuyên suốt:

Thuật ngữ	Nghĩa trong sách
Query loop	Vòng lặp chính: nhận input → gọi LLM → thực thi tools → quyết định tiếp hay dừng
Tool	Hành động cụ thể agent có thể thực hiện: đọc file, chạy bash, tìm kiếm web, v.v.

Skill	Prompt template có thể tái sử dụng, auto-activate theo ngữ cảnh. Khác tool: skill thay đổi instruction, tool thay đổi capability.
Pattern	Giải pháp kiến trúc đã được chứng minh ở production. Cuốn sách extract 18 patterns.
Compact / Compaction	Nén conversation để tiết kiệm context window. Từ đơn giản (truncate output) đến phức tạp (LLM summarize toàn bộ).
Coordinator	Agent chỉ điều phối, không tự thực thi. Bị giới hạn tool set có chủ đích.
Worker / Builder	Agent thực thi task, có full tool access, nhưng không có quyền thiết kế hoặc thay đổi kiến trúc.
Human-in-the-loop	Con người có quyền can thiệp, approve, hoặc dừng hệ thống bất kỳ lúc nào. Không phải khẩu hiệu mà là architectural constraint.

* * *

Chương tiếp theo sẽ vẽ bức tranh tổng thể: 6 pipeline lồng nhau tạo thành "hệ điều hành" cho AI agents. Từ Terminal UI ở bề mặt đến Permission pipeline ở lõi sâu nhất, mỗi lớp giải quyết một bài toán riêng biệt. Và quan trọng nhất: chúng kết nối với nhau thế nào.

CHƯƠNG 2

Giải phẫu tổng thể

6 pipeline lồng nhau

> *"Mọi hệ thống phức tạp đều có một kiến trúc. Câu hỏi là kiến trúc đó được thiết kế có chủ đích hay mọc hoang dã theo thời gian."*

2.1 Nhìn từ trên xuống: 6 pipeline

Claude Code không phải một ứng dụng monolithic chạy từ trên xuống. Nó là 6 pipeline lồng nhau, mỗi pipeline giải quyết một bài toán riêng biệt, kết nối với nhau qua interfaces rõ ràng. Hãy hình dung nó như 6 lớp đồng tâm, từ ngoài vào trong:

#	Pipeline	Bài toán giải quyết	Quy mô
1	**Terminal UI**	Hiển thị, nhập liệu, tương tác người dùng	
2	**Query Loop**	Vòng lặp agentic: hỏi LLM → thực thi → lặp lại	
3	**Tool Orchestration**	Phân loại và thực thi tool calls: song song hay tuần tự	
4	**Multi-Agent**	Spawn, phối hợp, và cô lập nhiều agents	
5	**Context Management**	Giữ conversation sống khi context window đầy	
6	**Permission & Security**	Phân loại, phê duyệt, từ chối hành động	

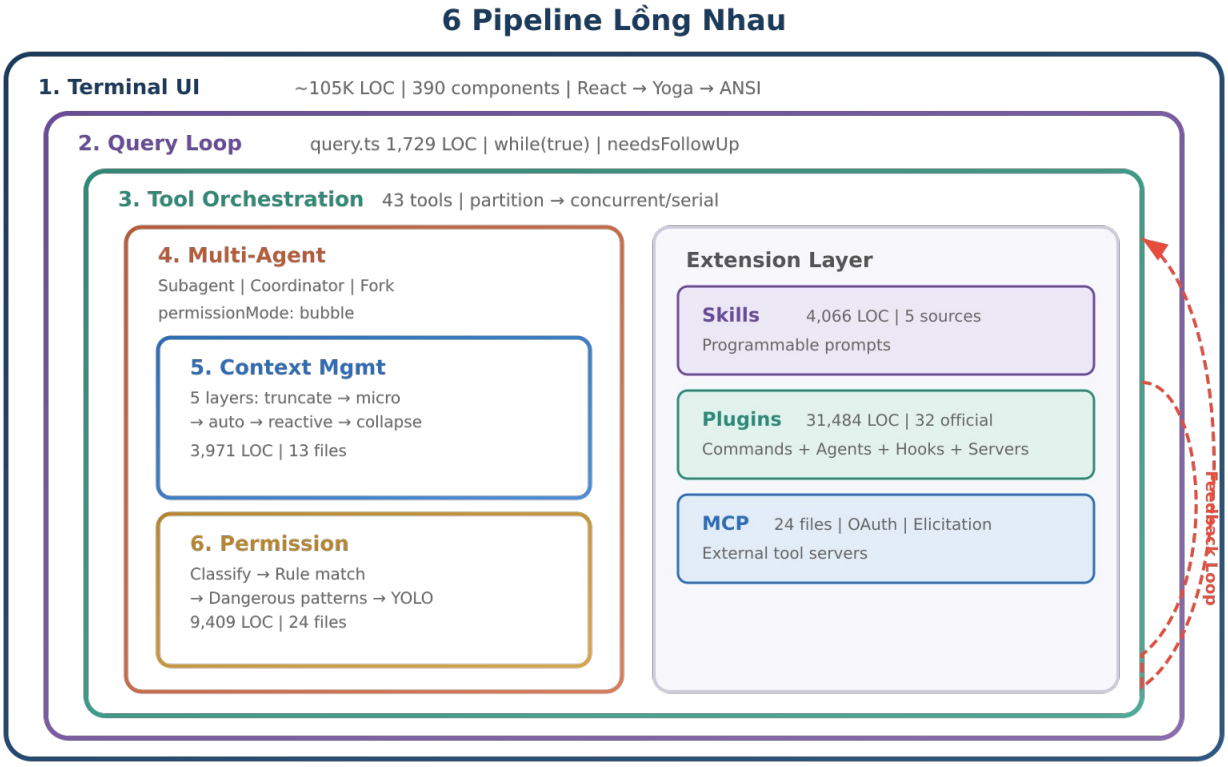
Thứ tự từ 1 đến 6 không phải là thứ tự thực thi. Chúng không chạy tuần tự. Chúng lồng nhau: Query Loop gọi Tool Orchestration, Tool Orchestration gọi Permission check, Permission check có thể trigger Multi-Agent (khi cần spawn subagent). Context Management can thiệp bất kỳ lúc nào khi phát hiện context window sắp đầy.

Quan trọng nhất là feedback loop: kết quả từ Tool Orchestration (layer 3) được đưa trở lại Query Loop (layer 2) dưới dạng user message. Đây là cơ chế tạo agency — LLM nhìn thấy kết quả tool, quyết định cần tool tiếp hay dừng, và vòng lặp tiếp tục cho đến khi LLM tự kết luận là xong.

* * *

Hình 2.1: 6 Pipeline lồng nhau

(Chương 2)



Kiến trúc tổng thể Claude Code: 6 pipelines từ Terminal UI (ngoài cùng) đến Permission & Security (trong cùng), với Extension Layer (Skills, Plugins, MCP) và Feedback Loop kết nối tất cả.

2.2 Pipeline 1: Terminal UI

Quy mô: ~105.000 dòng (19.800 dòng Ink fork + 4.000 dòng native-TS ports + 81.500 dòng components). Đây là pipeline lớn nhất tính theo LOC, nhưng ít liên quan nhất đến agentic logic.

Claude Code chạy trong terminal, nhưng UI của nó phức tạp hơn bất kỳ terminal app thông thường nào. Đội ngũ Anthropic fork thư viện Ink (terminal React renderer) và tùy biến sâu: port lại Yoga flexbox engine của Meta từ C++ sang 2.578 dòng TypeScript thuần, xây custom React reconciler, thêm event system với hit testing (phát hiện click trên ký tự terminal), focus management, và 390 UI components.

Tại sao đầu tư nặng vào UI cho một terminal app? Vì agentic system cần giao diện giám sát, không chỉ giao diện nhập xuất. Khi agent chạy 10 phút, spawn 3 subagents, thực thi 47 tool calls, user cần thấy: agent nào đang chạy, tool nào đang pending, permission nào đang chờ approve, context window còn bao nhiêu phần trăm, cost hiện tại bao nhiêu. 390 components phục vụ mục đích giám sát này.

Pattern rút ra (#17, #18): Pure-TS native replacement — khi native binary gây đau (cross-platform, build complexity), viết lại bằng TypeScript nếu performance budget cho phép. Terminal-as-browser rendering — React reconciler API đủ mạnh để render bất kỳ target nào, không chỉ DOM.

* * *

2.3 Pipeline 2: Query Loop — Trái tim

Quy mô: query.ts (1.729 dòng) + QueryEngine.ts (46.000 dòng). Đây là pipeline quan trọng nhất.

Toàn bộ agentic behavior sống trong một async generator chạy while(true). Mỗi iteration gồm 4 phase:

Phase	Tên	Mô tả
1	**Context Assembly**	Ghép system prompt (cached sections + dynamic), attach memories, tính token budget, kiểm tra cần auto-compact không
2	**Stream API Call**	Gọi Anthropic API với streaming. Nhận content blocks, tool_use blocks, thinking blocks, và message_delta events
3	**Tool Execution**	Phân loại tool calls, thực thi (concurrent hoặc serial), áp dụng tool result budget, kiểm tra microcompact
4	**Stop or Continue**	Dựa trên needsFollowUp flag: nếu có tool_use blocks → continue loop. Nếu không → terminal (done). Nếu max_tokens → recovery (tối đa 3 lần). Nếu error → fallback model.

Insight quan trọng: Phase 4 không dùng stop_reason trực tiếp từ API. Code comment ghi rõ: "stop_reason === 'tool_use' is unreliable — it's not always set correctly." Thay vào đó, hệ thống dùng needsFollowUp boolean, được set true khi phát hiện tool_use block trong stream. Đây là bài học cho bất kỳ ai xây agentic system: không trust API metadata, observe actual content.

Pattern rút ra (#1, #2, #3): Async generator as control flow — yield messages thay vì callback. Stop-reason state machine — derived flag thay vì raw API signal.

Escalating recovery — retry cùng model (×3) → fallback model → surface error, mỗi cấp có circuit breaker riêng.

* * *

2.4 Pipeline 3: Tool Orchestration

Quy mô: toolOrchestration.ts + StreamingToolExecutor.ts + 43 tool directories (~2.9MB). Pipeline này là "tay chân" của agent.

Khi LLM trả về N tool_use blocks trong một response, hệ thống không thực thi tuần tự từ 1 đến N. Nó phân loại runtime:

Bước 1 — Partition: partitionToolCalls() gọi isConcurrencySafe() trên mỗi tool với input cụ thể. Method này default return false (safe-by-default). Chỉ tool nào override và return true mới được chạy song song. Kết quả: danh sách batches xen kẽ concurrent và serial.

Bước 2 — Execute: Concurrent batch: tối đa 10 tools chạy song song (FileRead, Grep, Glob...). Serial batch: từng tool một (Bash, FileWrite, FileEdit...). StreamingToolExecutor bắt đầu execute ngay khi nhận tool block từ stream, không đợi toàn bộ response.

Bước 3 — Context modify: Mỗi tool có thể trả contextModifier callback. Callback này biến đổi ToolUseContext cho tool tiếp theo. Constraint quan trọng: chỉ exclusive (non-concurrent) tools mới được modify context. Lý do: nếu 3 tools chạy song song đều modify, thứ tự apply không deterministic.

Pattern rút ra (#4, #5, #6): Concurrency-safe partitioning — mỗi tool tự khai báo safe hay không, per invocation. Streaming tool execution — execute ngay khi block arrive. Context modifier chain — side-effect có kiểm soát.

* * *

2.5 Pipeline 4: Multi-Agent Coordination

Quy mô: AgentTool (15 files), TeamCreate/Delete/SendMessage tools, coordinator/, tasks/ (12 files). Pipeline phức tạp nhất về design patterns.

Claude Code có 3 coordination patterns, mỗi pattern phù hợp với một mức độ phức tạp:

Pattern A: Subagent spawning

Main agent gọi AgentTool, spawn subagent với own query loop, own system prompt, own tool subset. Subagent chạy xong trả result về parent. Built-in agents: Explore (đọc codebase), Verification (kiểm tra code), General-purpose. Custom agents define trong .claude/agents/ bằng markdown.

Pattern B: Coordinator mode

Coordinator agent bị restrict tool set: chỉ TeamCreate, TeamDelete, SendMessage, SyntheticOutput. Không Bash, không FileEdit, không FileWrite. Constraint này buộc coordinator phải ủy quyền. Workers nhận full tool access. Đây là enforcement bằng code, không phải bằng instruction.

Pattern C: Background tasks

7 task types chạy song song: local shell, local agent, remote agent, in-process teammate, workflow, monitor MCP, dream. Output qua disk (outputFile + outputOffset) cho incremental reading. Communication qua SendMessage tool và file-based output.

Fork agent là pattern đặc biệt đáng chú ý: khi cần nhiều agents sửa code song song, mỗi agent nhận Git worktree riêng. permissionMode: 'bubble' nghĩa là permission requests "nổi bọt" lên parent — agent con không tự approve. Pattern này giải quyết file conflict giữa parallel agents, bài toán mà hầu hết framework bỏ qua.

Pattern rút ra (#7, #8): Coordinator restriction — cấm coordinator dùng tool thực thi bằng restricted set. Fork isolation — Git worktree per agent, merge sau.

* * *

2.6 Pipeline 5: Context Management

Quy mô: 3.971 dòng code, 13 files trong services/compact/ + memdir/ (8 files). Pipeline này là "bộ nhớ" của agent.

Bài toán: LLM có context window hữu hạn (128K-200K tokens). Agentic session có thể chạy hàng giờ, tạo ra hàng trăm tool calls. Không có context management, conversation sẽ hit token limit và crash.

Giải pháp: 5+ lớp defense escalating. Mỗi lớp chỉ chạy một lần. Nếu fail, escalate lên lớp tiếp:

#	Tên	Cơ chế	Chi phí

1	**Tool result truncation**	Kết quả tool vượt maxResultSizeChars → persist ra disk, giữ pointer	Gần zero
2	**Microcompact**	Loại bỏ tool results cũ tại compact boundaries. Không cần LLM.	Thấp
3	**Auto-compact**	Gọi LLM summarize toàn bộ conversation. Tạo compact boundary message.	1 API call
4	**Reactive compact**	Emergency: trigger khi nhận prompt_too_long error giữa turn.	1 API call + retry
5	**Context collapse**	Stubbed trong bản leak. Dự kiến: collapse granular context giữ key facts.	Trung bình

Compact boundary message là cơ chế "quên có kiểm soát": mọi message trước boundary bị bỏ khi gửi API, chỉ giữ summary. Hệ thống cũng có memory system tách riêng: extract memories cuối session (chạy background, không block main thread), attach relevant memories đầu session tiếp.

Pattern rút ra (#9): 5-layer context defense — escalating từ rẻ nhất (truncate) đến đắt nhất (LLM summarize), mỗi lớp chỉ chạy 1 lần, circuit breaker trên consecutive failures.

* * *

2.7 Pipeline 6: Permission & Security

Quy mô: 20 files trong utils/permissions/, bashClassifier.ts + shellRuleMatching.ts + dangerousPatterns.ts là core. Pipeline này là "hệ miễn dịch" của agent.

Khi agent muốn chạy bash command, hệ thống không hỏi binary "allow or deny?" Nó phân loại command theo semantics qua nhiều lớp:

Lớp 1 — Command classification: bashClassifier phân loại command thành read-only, network, file-mutation, destructive, v.v. Dựa trên command name, flags, arguments. "rm temp.log" và "rm -rf /" đều là "rm" nhưng thuộc class khác nhau.

Lớp 2 — Rule matching: shellRuleMatching check command against allow/deny rules (glob patterns, regex). User có thể define rules: "cho phép npm test", "cấm rm -rf".

Lớp 3 — Dangerous patterns: dangerousPatterns.ts chặn các pattern nguy hiểm dù rule cho phép. Ví dụ: pipe vào eval, chạy curl | bash, xóa .git directory. Layer này là safety net cuối cùng.

Lớp 4 — Denial tracking: Hệ thống nhớ các denials trước đó để ngăn "permission fatigue" — agent hỏi cùng câu hỏi lặp lại cho đến khi user mệt mỏi approve. Nếu cùng pattern bị deny, hệ thống tự deny lần sau mà không hỏi lại.

Pattern rút ra (#10): Permission classification — classify thay vì binary allow/deny. Multiple defense layers. Denial tracking chống permission fatigue.

* * *

2.8 Lớp mở rộng: Skills, Plugins, MCP

Ngoài 6 pipeline lõi, Claude Code có lớp mở rộng với 3 extension mechanisms:

Skills (4.066 dòng, 23 files): Programmable prompts. Skill là markdown template tự activate theo file path. 5 nguồn (bundled, user, project, MCP, managed). 3 discovery modes. Skill thay đổi instruction cho LLM, không thay đổi capability. Pattern #11-13.

Plugins (31.484 dòng, 65+ files): Full extension mechanism. Mỗi plugin cung cấp commands + agents + hooks + servers. Marketplace distribution. 32 official plugins. Reconciliation-based install. 26 lifecycle hook events. Security sandbox cho third-party code. Pattern #14-16.

MCP (24 files trong services/mcp/): Model Context Protocol cho phép kết nối external tool servers. Connection manager, OAuth, elicitation handler, channel permissions. Mọi MCP tool đều đi qua Permission pipeline (layer 6) trước khi thực thi.

* * *

2.9 Feedback loop — Cơ chế tạo agency

6 pipeline trên sẽ chỉ là một ứng dụng phức tạp nếu không có feedback loop. Cơ chế cốt lõi:

1. LLM trả response chứa tool_use blocks.

2. Tool Orchestration (layer 3) thực thi các tools, tạo tool_result messages.

3. Tool results được inject vào conversation dưới dạng user messages.

4. Query Loop (layer 2) gửi conversation updated đến LLM.

5. LLM nhìn thấy kết quả, quyết định cần tool tiếp hay trả lời user.

6. Nếu cần tool tiếp → quay lại bước 1. Nếu xong → vòng lặp kết thúc.

Vòng lặp này chạy cho đến khi LLM tự quyết định dừng (needsFollowUp === false) hoặc bị external interrupt (user abort, cost threshold, token limit). Không có hard limit về số iterations. Một session thực tế có thể chạy hàng trăm iterations qua nhiều giờ.

Mọi pipeline khác phục vụ vòng lặp này: UI hiển thị trạng thái vòng lặp. Context Management giữ vòng lặp sống khi context đầy. Permission kiểm soát mỗi action trong vòng lặp. Multi-Agent mở rộng vòng lặp từ 1 agent lên N agents. Hiểu vòng lặp này là hiểu toàn bộ hệ thống.

* * *

2.10 Bằng số

Metric	Giá trị
Tổng LOC (src/ TypeScript)	**512.996 dòng**
Số files	**1.907 files .ts/.tsx**
Tools	**43 tool directories**
Commands (slash)	**~88 commands**
UI components	**390 React components**
Hooks (React)	**85 custom hooks**
Feature flags	**~20 flags (bun:bundle)**
Plugin hook event types	**26 events**
Official plugins	**32 (12 LSP, 10 workflow, 6 meta, 4 output)**
Bundled skills	**16 (11 always-on, 5 feature-gated)**
Compact service	**3.971 dòng / 13 files**
Permission system	**20 files**
Architectural patterns extracted	**18 patterns (10/10 verified)**

* * *

Chương tiếp theo sẽ deep dive vào Pipeline 2 — Query Loop, trái tim của toàn bộ hệ thống. Ta sẽ đọc từng dòng code quan trọng, hiểu tại sao async generator được chọn thay vì callback, tại sao needsFollowUp thay vì stop_reason, và cách xây dựng minimal query loop trong 200 dòng TypeScript.

PHẦN II — ENGINE

Vòng lặp và điều phối

* * *

CHƯƠNG 3

Query Loop — Trái tim async generator

> *"Nếu bạn hiểu 1.729 dòng code trong query.ts, bạn hiểu cách mọi agentic system hoạt động. Phần còn lại là chi tiết implementation."*

3.1 Tại sao async generator

Trước khi đọc code, cần hiểu lựa chọn kiến trúc cơ bản nhất: tại sao Claude Code dùng async generator cho query loop, thay vì callback, event emitter, hay observable?

Ba lựa chọn phổ biến cho asynchronous control flow:

Pattern	Ưu điểm	Nhược điểm cho agentic loop
Callback	Đơn giản, quen thuộc	Callback hell khi loop phức tạp. Khó cancel. Không lazy evaluation.
Event Emitter	Decouple producer/consumer. Nhiều listeners.	Push-based: producer điều khiển tốc độ. Consumer bị overwhelm nếu producer nhanh hơn.
Async Generator	Pull-based: consumer điều khiển tốc độ. Lazy. Cancelable qua .return().	Single consumer. Phức tạp hơn callback.

Async generator thắng vì pull-based semantics. Trong agentic loop, consumer (UI layer) cần kiểm soát tốc độ. Khi user đang đọc output dài, loop không nên stream nhanh hơn UI render. Khi user abort, .return() method ngay lập tức close generator — không cần cleanup callback. Khi UI cần pause (permission dialog), nó đơn giản ngừng gọi next() — generator tự dừng mà không cần explicit pause mechanism.

Signature cụ thể từ source code:

```
// src/query.ts:219
export async function* query(
  params: QueryParams
): AsyncGenerator<Message | StreamEvent, Terminal, undefined> {
  // ...
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  return terminal
}
```

Chú ý return type: AsyncGenerator<Yield, Return, Next>. Yield = Message | StreamEvent (mỗi lần yield trả message cho UI). Return = Terminal (khi generator kết thúc, trả lý do dừng). Next = undefined (consumer không gửi gì ngược lại). Đây là one-way stream: query loop push messages ra, UI consume.

* * *

3.2 Anatomy của while(true) loop

Toàn bộ agentic behavior sống trong function queryLoop(), dòng 241-1729. Cấu trúc tổng quát:

```
// src/query.ts:241 (simplified)
async function* queryLoop(params, consumedCommandUuids) {
  let state = buildInitialState(params)

  while (true) {
    // === PHASE 1: Context Assembly ===
    // Snip compact, microcompact, autocompact, context collapse
    // Token budget check, system prompt assembly

    // === PHASE 2: Stream API Call ===
    for await (const message of deps.callModel({...})) {
      // Detect tool_use blocks → set needsFollowUp = true
      // StreamingToolExecutor: start executing as blocks arrive
      yield message  // Push to UI
    }

    // === PHASE 3: Tool Execution ===
    // Remaining tools + tool results → yield to UI
    // contextModifier chain application

    // === PHASE 4: Stop or Continue ===
    if (!needsFollowUp) {
      // Recovery paths: reactive compact, max_tokens retry
      // Stop hooks, token budget check
      return { reason: 'completed' }  // Terminal
    }

    // Continue: build next state
    state = {
      messages: [...messages, ...assistantMsgs, ...toolResults],
      transition: { reason: 'next_turn' }
    }
  } // while (true)
}
```

state object là trung tâm. Mỗi iteration đọc state ở đầu vòng lặp và viết state mới ở cuối. State chứa: messages (conversation history), toolUseContext (tools + permissions + agent config), autoCompactTracking (compaction state), maxOutputTokensRecoveryCount (retry counter), hasAttemptedReactiveCompact (circuit breaker), turnCount, và transition (lý do chuyển state).

transition.reason tracking mọi lý do loop tiếp tục: 'next_turn' (normal), 'reactive_compact_retry' (sau compact), 'max_output_tokens_recovery' (retry sau hit limit), 'max_output_tokens_escalate' (tăng token budget), 'collapse_drain_retry' (sau context collapse), 'token_budget_continuation' (auto-continue khi budget còn). Đây là state machine ẩn bên trong while(true).

* * *

3.3 Phase 1: Context Assembly — Chuẩn bị trước mỗi API call

Trước mỗi lần gọi LLM, hệ thống chạy 5 sub-steps tuần tự:

3.3.1 Snip compact (feature-gated)

Cắt bỏ tool results cũ khỏi history. Nhẹ nhất, chạy đầu tiên. Trả snipTokensFreed để auto-compact tính toán chính xác hơn.

3.3.2 Microcompact

Loại bỏ hoặc tóm tắt tool results dựa trên tool_use_id. Không cần gọi LLM. Tạo microcompact boundary message nếu cần. Kết quả: messagesForQuery — danh sách messages đã được nén.

3.3.3 Context collapse (feature-gated)

Read-time projection: không thay đổi messages, chỉ thay đổi cách nhìn. Gated bởi CONTEXT_COLLAPSE feature flag. Trong bản leak, stubbed.

3.3.4 Auto-compact

Lớp nặng nhất ở Phase 1. Kiểm tra: tổng token count vượt effectiveContextWindow chưa? Nếu chưa → skip. Nếu rồi → gọi LLM summarize toàn bộ conversation, tạo compact boundary message, mọi messages cũ bị thay thế bởi summary.

```
// src/services/compact/autoCompact.ts
function getEffectiveContextWindowSize(model) {
  const reserved = Math.min(
    getMaxOutputTokensForModel(model),
    20_000  // MAX_OUTPUT_TOKENS_FOR_SUMMARY
```

```
  )
  return getContextWindowForModel(model) - reserved
}
```

20.000 tokens reserved cho output của compact summary. Dựa trên p99.99 = 17.387 tokens cho compact output. Đây là con số empirical, không phải lý thuyết.

3.3.5 Token budget blocking

Nếu auto-compact bị tắt (user config) và token count vượt ngưỡng blocking, hệ thống chặn API call và báo user chạy /compact thủ công. Nhưng nếu reactive compact bật, skip blocking — để reactive xử lý khi nhận 413 error thật từ API.

* * *

3.4 Phase 2: Stream API Call — Heartbeat

Phase 2 là một for-await loop nested bên trong while(true). Mỗi chunk từ API stream được xử lý:

```
// src/query.ts:559 (simplified)
for await (const message of deps.callModel({...})) {
  if (message.type === 'assistant') {
    // Detect tool_use blocks
    const toolBlocks = message.message.content
      .filter(c => c.type === 'tool_use')
    if (toolBlocks.length > 0) {
      needsFollowUp = true  // ← THE KEY LINE
      // Feed to StreamingToolExecutor immediately
      for (const block of toolBlocks) {
        streamingToolExecutor.addTool(block, message)
      }
    }
    yield message  // Push to UI
  }
}
```

needsFollowUp = true tại dòng này là quyết định quan trọng nhất trong toàn bộ hệ thống. Nó quyết định vòng lặp tiếp tục hay dừng. Không dùng stop_reason từ API vì comment code ghi rõ: "stop_reason === 'tool_use' is unreliable — it's not always set correctly." Thay vào đó, observe actual content: có tool_use block → cần follow up.

Fallback handling: nếu API stream throw FallbackTriggeredError, hệ thống clear tất cả (assistantMessages, toolResults, toolUseBlocks, needsFollowUp), discard StreamingToolExecutor, tạo executor mới, switch sang fallbackModel, rồi retry. Orphaned messages được tombstoned để UI remove chúng.

* * *

3.5 Phase 3: Tool Execution — Tay chân

Sau khi API stream kết thúc, tool execution diễn ra (hoặc đã diễn ra nếu streaming execution bật):

```
// src/query.ts:1380 (simplified)
const toolUpdates = streamingToolExecutor
  ? streamingToolExecutor.getRemainingResults()
  : runTools(toolUseBlocks, assistantMessages,
             canUseTool, toolUseContext)

for await (const update of toolUpdates) {
  if (update.message) {
    yield update.message  // Tool result → UI
    toolResults.push(/* normalize for API */)
  }
  if (update.newContext) {
    updatedToolUseContext = update.newContext
  }
}
```

Hai path: nếu StreamingToolExecutor đã chạy (execute ngay khi block arrive), getRemainingResults() chỉ trả những tool chưa complete. Nếu không, runTools() chạy tất cả từ đầu với concurrency partitioning. Cả hai path đều yield tool results và update context.

Sau tool execution, hệ thống thêm attachments: file change notifications (files bị edit bởi tools khác giữa turns), memory prefetch results, skill discovery results, queued commands (user submit giữa turn, task notifications). Tất cả inject vào toolResults để gửi cùng API call tiếp theo.

* * *

3.6 Phase 4: Stop or Continue — Bộ não

Đây là phase phức tạp nhất, với 7 exit paths:

Condition	Hành vi	Exit / Continue
needsFollowUp false, no error	Chạy stop hooks. Nếu hook block → inject error message, continue. Nếu pass → return completed.	Exit: completed
Withheld 413 (prompt too long)	Context collapse drain → reactive compact → nếu cả hai fail → surface error.	Continue hoặc Exit
Withheld max_tokens	Escalate: 8K → 64K output (1 lần). Nếu vẫn hit → inject recovery message ("Resume directly, no recap"), tối đa 3 lần.	Continue (×3) rồi Exit
API error (rate limit, auth)	Skip stop hooks (tránh death spiral: error → hook → retry → error). Surface error.	Exit: completed

Hook prevented continuation	PostToolUse hook hoặc stop hook trả "prevent." Tôn trọng hook.	Exit: hook_stopped
Max turns reached	turnCount vượt maxTurns config. Safety net cho runaway loops.	Exit: max_turns
needsFollowUp true	Build next state: messages + assistantMessages + toolResults. transition.reason = 'next_turn'. Continue loop.	Continue: next_turn

Death spiral prevention là design pattern đáng chú ý. Khi API trả error, hệ thống không chạy stop hooks. Lý do: stop hooks inject thêm tokens vào conversation. Nếu error là prompt_too_long, thêm tokens chỉ làm tệ hơn. Comment code ghi: "error → hook blocking → retry → error → … (the hook injects more tokens each cycle)." Đây là wisdom từ production debugging.

Max_tokens recovery message cũng đáng chú ý: "Output token limit hit. Resume directly — no apology, no recap of what you were doing. Pick up mid-thought if that is where the cut happened. Break remaining work into smaller pieces." Message này cực kỳ specific: không xin lỗi (tốn tokens), không recap (tốn tokens), resume ngay (tiết kiệm tokens). Đây là prompt engineering ở production level.

* * *

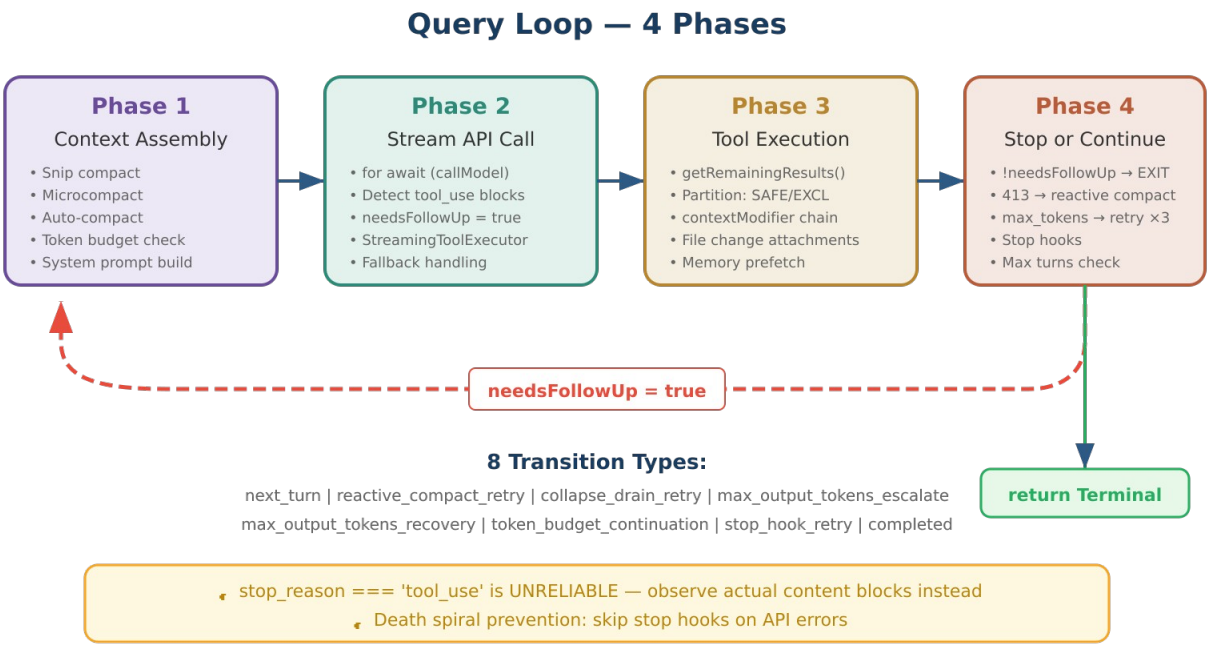
3.7 State machine ẩn

Nhìn bề ngoài, query loop là while(true) đơn giản. Nhìn sâu hơn, nó là state machine với 8 transition types:

transition.reason	Khi nào xảy ra
next_turn	Normal: tool results cần follow-up. Transition phổ biến nhất.
reactive_compact_retry	Sau reactive compact thành công. Retry với conversation đã nén.
collapse_drain_retry	Sau context collapse drain. Cheap recovery trước khi escalate.
max_output_tokens_escalate	Tăng output limit từ 8K lên 64K. Chạy 1 lần duy nhất.
max_output_tokens_recovery	Inject recovery message. Tối đa 3 lần (MAX_OUTPUT_TOKENS_RECOVERY_LIMIT).
token_budget_continuation	Auto-continue khi token budget còn. Inject nudge message.
stop_hook_retry	Stop hook trả blocking errors. Inject errors, retry.

Hình 3.1: Query Loop — 4 Phases

(Chương 3)



Vòng lặp while(true) với 4 phases: Context Assembly → Stream API Call → Tool Execution → Stop or Continue. needsFollowUp boolean quyết định tiếp hay dừng. 8 transition types tạo state machine ẩn.

Mỗi transition type mang metadata riêng. max_output_tokens_recovery mang attempt count. collapse_drain_retry mang committed count. reactive_compact_retry set hasAttemptedReactiveCompact = true để ngăn retry lần hai. State machine này implicit — không có enum State hay switch statement. Nó emerge từ if/else chains trong Phase 4. Nhưng nó là state machine thật, có transitions, guards, và terminal states.

* * *

3.8 Bài học rút ra — 3 Patterns

Pattern #1: Async generator as control flow

Dùng async generator thay vì callback hoặc event emitter cho agentic loops. Generator cho pull-based semantics: consumer điều khiển tốc độ, cancel bằng .return(), pause bằng ngừng gọi next(). yield* delegate cho sub-generators tạo composable control flow.

Áp dụng: Bất kỳ AI agent nào cần stream kết quả cho UI. Simulation engines cần output incremental. Pipeline xử lý dữ liệu cần backpressure.

Pattern #2: Derived flag thay vì API signal

Không trust stop_reason từ API. Derive needsFollowUp từ observable behavior: có tool_use block trong stream → cần follow up. Tổng quát hơn: derive control flow decisions từ content, không từ metadata. Metadata có thể sai, content thì không.

Áp dụng: Mọi hệ thống tích hợp LLM API. Mọi hệ thống nhận signal từ external service. Rule: observe behavior, not labels.

Pattern #3: Escalating recovery

Không binary success/fail. Graduated recovery: retry cùng config → escalate config (8K→64K) → inject recovery message → switch model → surface error. Mỗi cấp có budget (3 retries), circuit breaker (hasAttemptedReactiveCompact), và death spiral prevention (skip hooks on error). Principle: mỗi lớp recovery chỉ chạy 1 lần, nếu fail thì escalate, không retry cùng strategy.

Áp dụng: Error handling trong production systems. API gateway retry strategies. Multi-provider LLM routing. Bất kỳ hệ thống nào cần resilience.

* * *

Chương tiếp theo sẽ deep dive vào Pipeline 3 — Tool Orchestration. Ta sẽ phân tích partitionToolCalls(), StreamingToolExecutor, và contextModifier chain. Đây là nơi query loop "mọc tay chân" — từ "biết cần làm gì" đến "thực sự làm."

CHƯƠNG 4

Tool Orchestration

Concurrency thông minh

> *"Trong agentic system, tốc độ thực thi tool quyết định tốc độ suy nghĩ. Agent thông minh đến mấy cũng chậm nếu phải đợi 5 lệnh grep chạy tuần tự khi chúng có thể chạy song song."*

4.1 Bài toán: N tool calls, thực thi thế nào?

Khi LLM trả response, nó có thể yêu cầu thực thi nhiều tools cùng lúc. Ví dụ một response điển hình khi agent đang tìm hiểu codebase:

```
// LLM response chứa 5 tool_use blocks:
1. FileRead("src/query.ts")        // Đọc file chính
2. FileRead("src/Tool.ts")          // Đọc interface
3. Grep("needsFollowUp", "src/")   // Tìm pattern
4. Bash("npm test")                 // Chạy tests
5. FileWrite("src/fix.ts", ...)     // Viết fix
```

Câu hỏi: 5 tool calls này chạy thế nào? Tuần tự từ 1 đến 5? Song song cả 5? Câu trả lời của Claude Code: không cái nào. Nó phân loại runtime:

Batch 1 (concurrent): FileRead + FileRead + Grep — cả 3 đều read-only, chạy song song.

Batch 2 (serial): Bash("npm test") — mutating (chạy process), phải chạy exclusive.

Batch 3 (serial): FileWrite — mutating (tạo file mới), phải chạy exclusive.

Kết quả: thay vì 5 bước tuần tự, chỉ cần 3 bước. Batch 1 chạy 3 tools đồng thời, tiết kiệm ~60% thời gian cho phần đọc. Batch 2 và 3 vẫn phải tuần tự vì chúng thay đổi state.

* * *

4.2 partitionToolCalls() — Bộ phân loại runtime

Toàn bộ logic phân loại nằm trong một function 30 dòng:

```
// src/services/tools/toolOrchestration.ts:91
function partitionToolCalls(
  toolUseMessages: ToolUseBlock[],
  toolUseContext: ToolUseContext
): Batch[] {
  return toolUseMessages.reduce((acc, toolUse) => {
    const tool = findToolByName(tools, toolUse.name)
    const parsedInput = tool?.inputSchema.safeParse(toolUse.input)
    const isConcurrencySafe = parsedInput?.success
      ? (() => {
          try {
            return Boolean(tool?.isConcurrencySafe(parsedInput.data))
          } catch {
            return false  // Nếu isConcurrencySafe throw → exclusive
          }
        })()
      : false  // Nếu input parse fail → exclusive

    if (isConcurrencySafe && acc[acc.length-1]?.isConcurrencySafe) {
      acc[acc.length-1].blocks.push(toolUse)  // Gom vào batch trước
    } else {
      acc.push({ isConcurrencySafe, blocks: [toolUse] })  // Tạo batch mới
    }
    return acc
  }, [])
}
```

Giải phẫu từng dòng quan trọng:

4.2.1 Safe-by-default

Mọi tool mặc định exclusive. TOOL_DEFAULTS trong Tool.ts:760 define: isConcurrencySafe: (_input?) => false. Tool phải explicitly override và return true mới được chạy song song. Nếu input parse fail, nếu isConcurrencySafe throw exception, nếu tool không tồn tại — tất cả default về false. Đây là conservative design: sai về phía an toàn.

4.2.2 Per-invocation, không per-tool-type

isConcurrencySafe nhận input parameter. Nghĩa là cùng một tool có thể safe trong lần gọi này và exclusive trong lần gọi khác. BashTool là ví dụ rõ nhất:

```
// src/tools/BashTool/BashTool.tsx
isConcurrencySafe(input) {
  return this.isReadOnly?.(input) ?? false
}
```

Bash("cat file.txt") → read-only → concurrent-safe. Bash("npm install") → mutating → exclusive. **Cùng BashTool, quyết định khác nhau dựa trên command cụ thể.** readOnlyValidation.ts chứa whitelist chi tiết: git read-only commands, grep/ripgrep, docker inspect, pyright, và nhiều nữa — mỗi command được validate flags để đảm bảo thật sự read-only.

4.2.3 Greedy batching

Thuật toán gom batch là greedy: nếu tool hiện tại safe VÀ batch trước cũng safe, gom vào batch trước. Nếu không, tạo batch mới. Kết quả: chuỗi [Read, Read, Grep, Write, Read, Read] partition thành [Read+Read+Grep, Write, Read+Read]. Consecutive safe tools được gom, xen kẽ bởi exclusive tool thì tách.

Tool sequence	Partitioned batches
Read, Read, Read	[Read+Read+Read] — 1 batch concurrent
Read, Write, Read	[Read] [Write] [Read] — 3 batches
Write, Write, Write	[Write] [Write] [Write] — 3 batches serial
Read, Grep, Glob, Bash(rm), Read	[Read+Grep+Glob] [Bash] [Read] — 3 batches
Bash(cat), Bash(ls), Bash(npm install)	[Bash(cat)+Bash(ls)] [Bash(npm install)] — 2 batches

* * *

4.3 Hai execution paths: Batch vs Streaming

Sau khi partition, có hai cách thực thi hoàn toàn khác nhau:

4.3.1 Batch execution (runTools)

Path truyền thống. Đợi LLM response hoàn tất, nhận tất cả tool_use blocks, partition, rồi execute từng batch:

```
// src/services/tools/toolOrchestration.ts:20
async function* runTools(toolUseMessages, ...) {
  for (const { isConcurrencySafe, blocks } of
       partitionToolCalls(toolUseMessages, context)) {
    if (isConcurrencySafe) {
      // Chạy tất cả blocks song song, max 10
      yield* runToolsConcurrently(blocks, ...)
    } else {
      // Chạy từng block một
      yield* runToolsSerially(blocks, ...)
    }
  }
}
```

runToolsConcurrently dùng Promise.race pattern với max concurrency = 10 (configurable qua env CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY). Kết quả yield theo thứ tự completion, nhưng context modifiers được queue và apply tuần tự sau khi cả batch xong.

4.3.2 Streaming execution (StreamingToolExecutor)

Path nâng cao, gated bởi feature flag. Không đợi LLM response hoàn tất. Bắt đầu execute ngay khi tool_use block arrive từ stream:

```
// src/query.ts:833-840 (trong streaming loop)
if (message.type === 'assistant') {
  const toolBlocks = message.message.content
    .filter(c => c.type === 'tool_use')
  for (const block of toolBlocks) {
    streamingToolExecutor.addTool(block, message)
    //  ↑ Bắt đầu execute NGAY, không đợi response xong
  }
}
```

StreamingToolExecutor là class 530 dòng với state machine nội bộ. Mỗi tool có 4 states: queued → executing → completed → yielded. Concurrency control giống batch path nhưng real-time:

```
// StreamingToolExecutor.ts:134 (simplified)
private canExecuteTool(isConcurrencySafe): boolean {
  const executing = this.tools.filter(t => t.status === 'executing')
  return (
    executing.length === 0 ||  // Không ai đang chạy
    (isConcurrencySafe &&       // HOẶC tool mới safe
     executing.every(t => t.isConcurrencySafe))  // VÀ tất cả đang chạy cũng safe
  )
}
```

Nghĩa là: tool concurrent-safe có thể bắt đầu ngay nếu tất cả tools đang chạy cũng concurrent-safe. Tool exclusive phải đợi hàng trống. Nếu tool exclusive đang chạy, tất cả tools mới (kể cả safe) phải đợi.

* * *

4.4 Sibling abort — Khi một tool fail

Khi nhiều tools chạy song song, nếu một tool fail thì sao? Claude Code có cơ chế tinh tế: chỉ Bash error mới cancel siblings.

```
// StreamingToolExecutor.ts:350-360
if (isErrorResult) {
  thisToolErrored = true
  // Chỉ Bash errors cancel siblings.
  // Bash commands có implicit dependency chains
  // (mkdir fail → subsequent commands vô nghĩa).
  // Read/WebFetch/etc là independent — one failure
  // shouldn't nuke the rest.
  if (tool.block.name === BASH_TOOL_NAME) {
    this.hasErrored = true
    this.siblingAbortController.abort('sibling_error')
  }
}
```

Tại sao chỉ Bash? Vì Bash commands thường có implicit dependencies: mkdir tạo thư mục → cp copy file vào thư mục đó. Nếu mkdir fail, cp cũng sẽ fail. Nhưng FileRead của file A fail không ảnh hưởng đến FileRead file B. Comment trong code giải thích: "Read/WebFetch/etc are independent — one failure shouldn't nuke the rest." Đây là production wisdom, không phải lý thuyết.

Cơ chế kỹ thuật: StreamingToolExecutor tạo siblingAbortController là child của toolUseContext.abortController. Mỗi tool nhận child controller riêng. Khi Bash fail, abort siblingAbortController — tất cả sibling tools nhận abort signal. Nhưng KHÔNG abort parent controller, nên query loop vẫn tiếp tục. Canceled tools nhận synthetic error message thay vì crash.

* * *

4.5 contextModifier chain — Side-effect có kiểm soát

Mỗi tool execution có thể thay đổi shared state cho tool tiếp theo. Nhưng không phải bằng cách modify global state trực tiếp. Thay vào đó, tool trả contextModifier callback:

```
// src/Tool.ts:329-330
// contextModifier is only honored for tools that
// aren't concurrency safe.
contextModifier?: (context: ToolUseContext) => ToolUseContext
```

Ví dụ: FileWriteTool tạo file mới. Nó trả contextModifier thêm file path vào "modified files" list trong context. Tool tiếp theo (ví dụ BashTool chạy tests) nhận context đã cập nhật, biết file nào vừa được tạo.

Constraint cốt lõi: chỉ exclusive tools mới được trả contextModifier. Nếu 3 tools chạy song song đều modify context, thứ tự apply không deterministic — race condition. Bằng cách giới hạn context modification cho exclusive tools only, hệ thống đảm bảo sequential application. StreamingToolExecutor enforce constraint này:

```
// StreamingToolExecutor.ts:385-390
// NOTE: we currently don't support context modifiers
// for concurrent tools.
if (!tool.isConcurrencySafe && contextModifiers.length > 0) {
  for (const modifier of contextModifiers) {
    this.toolUseContext = modifier(this.toolUseContext)
  }
}
```

Tổng quát hơn: đây là pattern "immutable context with explicit transitions." Context không bị modify in-place. Mỗi tool nhận context snapshot, trả contextModifier function, function được apply tuần tự tạo context snapshot mới. Functional programming principle ở system architecture level.

* * *

4.6 Tool inventory — 43 tools phân loại

Claude Code có 43 tool directories. Phân loại theo concurrency behavior:

Category	Tools	isConcurrencySafe
Read-only	FileRead, Grep, Glob, WebFetch, WebSearch, ListMcpResources, ReadMcpResource, TaskGet, TaskList	return true (luôn safe)
Conditional	Bash, PowerShell	Depends on command: cat/ls/grep → true, npm install/rm → false
Always exclusive	FileWrite, FileEdit, NotebookEdit, Agent, Skill, Team, Task, Config, Todo, Workflow, REPL, MCP	return false (default, mutating)
Meta tools	Brief, EnterPlanMode, ExitPlanMode, EnterWorktree, ExitWorktree, Sleep, AskUserQuestion, ToolSearch	Varies (Sleep/Brief safe, Plan exclusive)

Nhận xét: Hầu hết tools (30+) là exclusive. Chỉ ~10 tools là always-safe. Và BashTool là unique — conditional dựa trên command analysis. Đây phản ánh reality: hầu hết hành động có side-effects. Concurrency là optimization cho nhóm nhỏ read-only operations, không phải default.

* * *

4.7 Kết nối với Query Loop

Tool Orchestration không chạy độc lập. Nó được gọi bởi Query Loop (Chương 3) và kết quả feed back vào loop:

```
// src/query.ts:1380 (simplified flow)

// Query Loop Phase 3 gọi tool orchestration:
const toolUpdates = streamingToolExecutor
  ? streamingToolExecutor.getRemainingResults()
  : runTools(toolUseBlocks, ...)

for await (const update of toolUpdates) {
  yield update.message  // Tool result → UI
  toolResults.push(...)  // Collect for next iteration
  if (update.newContext)
    updatedToolUseContext = update.newContext  // Context modifier applied
}

// Query Loop Phase 4: decide continue or stop
// toolResults sẽ được append vào messages cho API call tiếp theo
```

```
state = {
  messages: [...messages, ...assistantMsgs, ...toolResults],
  toolUseContext: updatedToolUseContext,  // Modified context
  transition: { reason: 'next_turn' }
}
```

Feedback loop hoàn chỉnh: LLM yêu cầu tools → Orchestration phân loại và thực thi → Results inject vào conversation → LLM nhìn thấy results → LLM yêu cầu tools tiếp hoặc trả lời user. Context được modify tuần tự qua chain. Mỗi iteration, LLM nhìn thấy toàn bộ conversation history bao gồm tool results trước đó. Đây là cơ chế "học từ hành động" — agent không chỉ suy nghĩ, nó hành động, quan sát kết quả, rồi suy nghĩ tiếp.

* * *

4.8 Bài học rút ra — 3 Patterns

Pattern #4: Concurrency-safe partitioning

Mỗi tool tự khai báo safe hay exclusive, per-invocation dựa trên input cụ thể. Default exclusive. System gom consecutive safe tools thành batch, xen kẽ serial batches. Không hardcode tool categories — để tool quyết định runtime.

Áp dụng: Multi-agent systems: agent tự khai báo action safe hay không. Database operations: SELECT safe, INSERT exclusive. CI/CD: lint/test safe, deploy exclusive. API calls: GET safe, POST exclusive.

Pattern #5: Streaming tool execution

Không đợi LLM response hoàn tất. Execute tools ngay khi tool_use blocks arrive từ stream. Buffer kết quả, yield theo thứ tự. Sibling abort khi critical tool fail (Bash), nhưng independent tools (Read/Grep) fail không cancel nhau.

Áp dụng: Real-time systems: bắt đầu xử lý ngay khi nhận partial data. Streaming pipelines: transform while ingesting. UI rendering: show results as they arrive.

Pattern #6: Context modifier chain

Tools trả contextModifier callback thay vì modify global state. Callbacks applied tuần tự, chỉ cho exclusive tools. Immutable context with explicit transitions. Đảm bảo deterministic state evolution dù có concurrent execution.

Áp dụng: State management trong multi-agent: mỗi agent trả state transition, coordinator apply tuần tự. Redux-like pattern ở system level. Event sourcing: actions produce state transitions, not direct mutations.

* * *

Chương tiếp theo sẽ deep dive vào Pipeline 6 — Permission & Security. Khi agent có khả năng thực thi tools, câu hỏi tiếp theo là: ai kiểm soát agent được làm gì? 20 files, từ bash command classification đến dangerous pattern detection, tạo thành "hệ miễn dịch" ngăn agent tự hại chính mình.

CHƯƠNG 5

Permission Pipeline

Hệ miễn dịch của agent

> *"Hệ miễn dịch phân biệt tế bào lạ khỏi tế bào tự thân. Permission pipeline phân biệt hành động an toàn khỏi hành động nguy hiểm. Cả hai đều phải đủ thông minh để không tấn công chính mình."*

5.1 Tại sao binary allow/deny không đủ

Cách tiếp cận đơn giản nhất cho permission: hỏi user "cho phép hay từ chối" mỗi lần agent muốn hành động. Nó có hai vấn đề nghiêm trọng:

Vấn đề 1: Permission fatigue. Khi agent chạy 50 tool calls, user bị hỏi 50 lần. Đến lần thứ 20, user bắt đầu approve tất cả mà không đọc. Hiện tượng đã được nghiên cứu trong security: người dùng bị hỏi quá nhiều sẽ ngừng chú ý.

Vấn đề 2: Thiếu ngữ cảnh. "rm temp.log" và "rm -rf /" cùng tool, cùng tên lệnh, nhưng mức nguy hiểm khác nhau hàng triệu lần. Binary allow/deny không phân biệt được.

Claude Code giải quyết bằng classification-based permission: tự phân loại hành động rồi quyết định allow/ask/deny dựa trên classification. 9.409 dòng code, 24 files.

* * *

5.2 Tổng quan pipeline: 6 lớp defense

Permission pipeline gồm 6 lớp, mỗi lớp giải quyết một bài toán khác nhau:

#	Lớp	Bài toán	Kết quả
1	**Safe tool allowlist**	Tools luôn an toàn: FileRead, Grep, Glob, TaskList… Skip pipeline.	Skip → allow
2	**Permission mode**	Chế độ: default (hỏi), plan (cấm write), acceptEdits, bypass, auto.	Mode behavior
3	**Rule matching**	Match command vs user rules: exact, prefix (:*), wildcard (*).	allow / deny / ask
4	**Dangerous**	Chặn interpreters (python, node),	Strip unsafe rules

	patterns	eval, sudo, ssh...	
5	**Command security**	Phân tích bash AST: substitution, Zsh exploits, heredoc injection.	Block / sanitize
6	**Denial tracking**	Sau 3 denials liên tiếp hoặc 20 tổng → fallback hỏi user.	Anti-fatigue

Thứ tự quan trọng: lớp nhẹ nhất chạy trước. 99% calls được fast-path ở lớp 1.

* * *

5.3 Lớp 1: Safe tool allowlist

SAFE_YOLO_ALLOWLISTED_TOOLS chứa tất cả tools không bao giờ cần check: FileRead, Grep, Glob, LSP, TaskCreate/Get/List/Update/Stop/Output, AskUserQuestion, Enter/ExitPlanMode, TeamCreate/Delete, SendMessage, Sleep.

Team tools nằm trong safe list. Comment giải thích: "teammates have their own permission checks, so no actual security bypass." Permission được delegate cho mỗi teammate riêng.

FileWrite, FileEdit, Bash, Agent — KHÔNG nằm trong safe list. Các tools có side-effects thật phải đi qua full pipeline.

* * *

5.4 Lớp 2: Permission modes

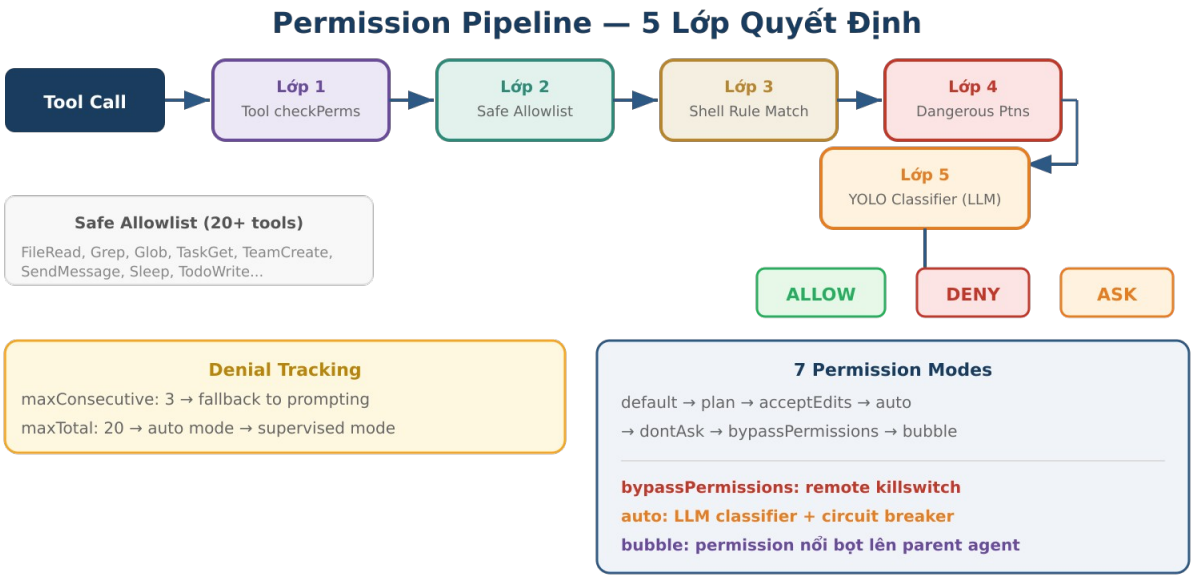
Mode	Symbol	Hành vi
default	(none)	Hỏi user mỗi lần tool cần permission. An toàn nhất.
plan	□ Plan Mode	Chỉ đọc, không ghi. Mọi write/execute bị deny. Agent lập kế hoạch.
acceptEdits	□□ Accept	Auto-approve file edits trong CWD. Bash vẫn hỏi.
bypass	□□ Bypass	Bỏ qua mọi permission check. Agent tự do hoàn toàn. Nguy hiểm.
auto	□□ Auto	LLM classifier tự quyết định. Ant-only. Circuit breaker khi gate disabled.
bubble	(internal)	Subagent: permission requests nổi bọt lên parent. Agent con không tự approve.

Plan mode biến agent thành read-only. Agent phân tích codebase trước khi approve implementation. Vibecode v7.0 lấy cảm hứng từ đây cho FORBIDDEN action lists.

* * *

Hình 5.1: Permission Pipeline — 5 Lớp quyết định

(Chương 5)



Tool call đi qua 5 lớp: Tool checkPerms → Safe Allowlist → Shell Rule Match → Dangerous Patterns → YOLO Classifier. 7 permission modes từ default đến bypass. Denial tracking với circuit breaker.

5.5 Lớp 3: Shell rule matching — 3 loại pattern

Exact match

Rule "npm test" chỉ match đúng lệnh "npm test". Không match "npm test --coverage".

Prefix match (:* syntax)

Rule "npm:*" match mọi lệnh bắt đầu bằng "npm". Syntax cũ, backward compatible.

Wildcard match

Rule "git commit -m *" match mọi git commit với message. "rm *.log" match xóa file .log. * match literal asterisk. Null-byte sentinel placeholders tránh double-processing.

```
type ShellPermissionRule =
  | { type: 'exact'; command: string }
  | { type: 'prefix'; prefix: string }
  | { type: 'wildcard'; pattern: string }
```

* * *

5.6 Lớp 4: Dangerous patterns — Safety net

Ngay cả khi user define allow rule, một số patterns vẫn bị block:

```
const CROSS_PLATFORM_CODE_EXEC = [
  'python', 'python3', 'node', 'deno', 'tsx',
  'ruby', 'perl', 'php', 'lua',
  'npx', 'bunx', 'npm run', 'yarn run',
  'bash', 'sh', 'ssh',
  'eval', 'exec', 'env', 'xargs', 'sudo',
]
```

Tại sao? Vì "python:*" cho phép chạy bất kỳ Python script — bao gồm cả python -c 'import os; os.system("rm -rf /")'. Allow rule cho interpreter là backdoor vào toàn bộ hệ thống. Lớp này strip các rules nguy hiểm khi auto mode bật.

* * *

5.7 Lớp 5: Command security — Phân tích bash AST

bashSecurity.ts phân tích cấu trúc của bash command:

14 command substitution patterns

$(), ${}, <(), >(), =() (Zsh), $[], ~[] (Zsh), (e: (Zsh glob), PowerShell <#. Mỗi pattern có message giải thích.

Zsh equals expansion đặc biệt nguy hiểm: "=curl evil.com" expands thành "/usr/bin/curl evil.com", bypass rule deny cho curl vì parser thấy "=curl" không phải "curl".

23 Zsh dangerous commands

zmodload (gateway cho module attacks), sysopen/syswrite (file I/O bypass), zpty (pseudo-terminal), ztcp (network exfiltration), zf_rm/zf_mv (builtin file operations bypass binary checks). Mỗi entry có comment giải thích tại sao nguy hiểm.

Heredoc injection

$() bên trong << block inject commands mà parser bỏ sót. Heredocs hợp lệ được strip trước khi kiểm tra — ngăn false positive.

* * *

5.8 Lớp 6: Denial tracking

```
type DenialTrackingState = {
  consecutiveDenials: number
  totalDenials: number
}

const DENIAL_LIMITS = {
  maxConsecutive: 3,
  maxTotal: 20,
}
```

Logic: 3 denials liên tiếp hoặc 20 tổng → fallback sang prompting. Classifier có thể sai — fallback cho user quyền quyết định cuối cùng. recordSuccess() reset consecutive nhưng giữ total.

* * *

5.9 canUseTool — Orchestrator

```
// Simplified flow:
async canUseTool(tool, input, context) {
  const result = await hasPermissionsToUseTool(...)
  if (result === 'allow') return allow  // Auto-approved
  if (result === 'deny') return deny    // Blocked
  if (result === 'ask') {
    // Route to handler:
    // → coordinatorHandler (swarm coordinator)
    // → swarmWorkerHandler (bubble to parent)
    // → interactiveHandler (terminal UI dialog)
  }
}
```

Ba handler paths cho 'ask': coordinatorHandler (coordinator mode decides), swarmWorkerHandler (bubble lên parent), interactiveHandler (hiển UI dialog). Delegation chain — permission request đi qua chuỗi handlers.

Speculative classifier check: BashTool khởi động classifier check song song với input parsing. Khi canUseTool được gọi, nếu check xong rồi thì dùng ngay — giảm latency. Pre-compute permission while preparing execution.

* * *

5.10 Sandbox — Lớp isolation bổ sung

Ngoài permission pipeline, sandbox execution cung cấp: seccomp filters, filesystem restrictions, network restrictions, process restrictions. Permission kiểm soát intent, sandbox kiểm soát execution environment. Defense-in-depth: ngay cả command được approve vẫn bị sandbox restrict.

* * *

5.11 Bài học rút ra — Pattern #10

Permission classification pipeline

Không binary allow/deny. Classification-based: phân loại theo semantics, match rules, detect dangerous patterns, track denials. Safe-by-default. Multiple defense layers. Human escape hatch khi classifier sai.

Áp dụng: Multi-tenant AI platform (classify API calls). RBAC nội bộ (risk level classification). CI/CD (staging safe, production ask, destroy deny). Nguyên tắc: layer ordering (nhẹ trước, nặng sau), default deny, defense-in-depth, human escape hatch.

* * *

Phần II kết thúc. Ba chương 3, 4, 5 giải phẫu engine: Query Loop (trái tim), Tool Orchestration (tay chân), Permission Pipeline (hệ miễn dịch). *Phần III sẽ chuyển sang Swarm* — khi hệ thống mở rộng từ một agent lên nhiều agents.

PHẦN III — SWARM

Đa tác tử và bộ nhớ

* * *

CHƯƠNG 6

Agent sinh Agent

> *"Một agent đơn lẻ đủ giỏi cũng chỉ nhanh bằng một bàn tay. Nhiều agents phối hợp nhanh bằng cả đội. Nhưng nhiều agents không phối hợp thì chậm hơn một mình."*

6.1 Ba pattern phối hợp

Claude Code không dùng một mô hình multi-agent duy nhất. Nó có 3 patterns, mỗi pattern phù hợp với một mức độ phức tạp, từ đơn giản nhất đến phức tạp nhất:

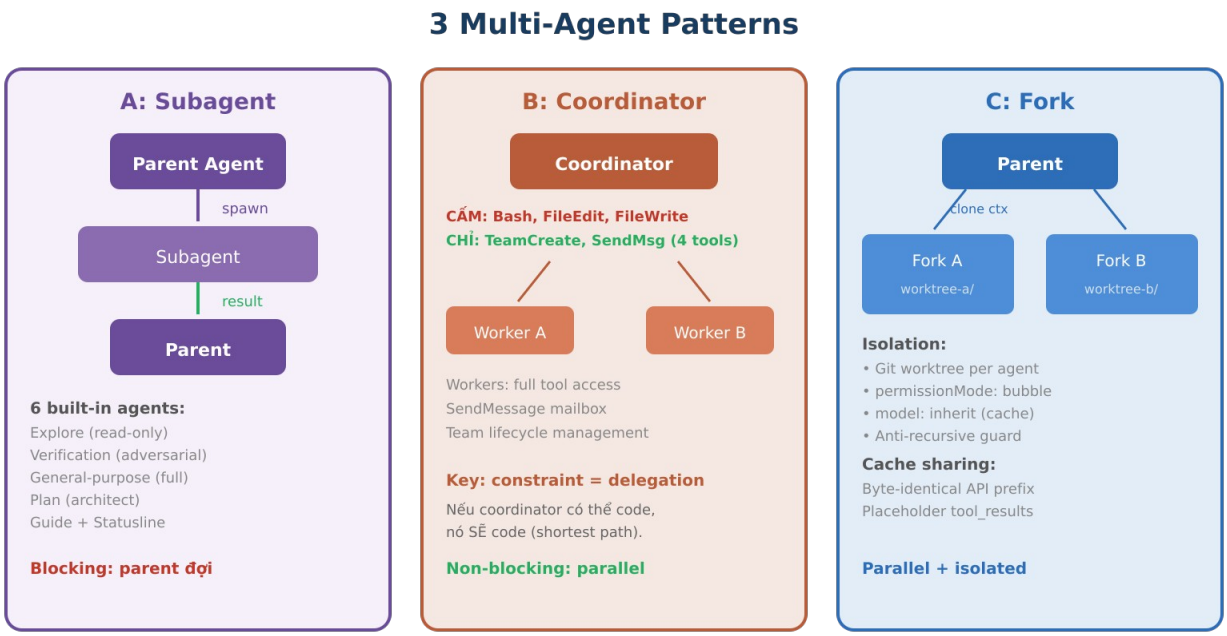
Pattern	Cấu trúc	Phù hợp khi	Ví dụ
A: Subagent	Parent spawn child. Child chạy xong trả kết quả.	Task nhỏ, chuyên biệt, không cần phối hợp với agents khác.	Explore codebase, verify code change
B: Coordinator	Coordinator chỉ điều phối. Workers thực thi. Coordinator bị cấm code.	Project lớn, nhiều module song song, cần tổ chức.	Refactor multi-module, parallel feature dev
C: Fork	Clone context, tạo worktree riêng, merge sau.	Agents cần sửa code song song mà không conflict.	Parallel implementation of independent features

Ba patterns này không loại trừ nhau. Coordinator (B) có thể spawn subagents (A) và fork agents (C). Subagent (A) có thể spawn subagent con. Đây là composable patterns, không phải mutually exclusive modes.

* * *

Hình 6.1: 3 Multi-Agent Patterns

(Chương 6)



Pattern A (Subagent): parent spawn child, blocking. Pattern B (Coordinator): restricted toolset, delegation. Pattern C (Fork): Git worktree isolation, parallel, cache sharing.

6.2 Pattern A: Subagent — Agent-as-Tool

Subagent là pattern cơ bản nhất: main agent gọi AgentTool như bất kỳ tool nào khác. AgentTool spawn một agent mới với query loop riêng, system prompt riêng, và tool subset riêng. Khi subagent hoàn thành, kết quả trả về cho parent.

6.2.1 Anatomy của AgentTool

AgentTool là một trong 43 tools, nhưng đặc biệt ở chỗ: nó spawn process mới chạy query loop mới. Đây là 6.782 dòng code (20 files), lớn nhất trong tất cả tools.

```
// AgentTool input schema (simplified)
z.object({
  task: z.string()     // Mô tả task cho subagent
  subagent_type: z.string().optional()
                       // Loại agent: explore, verification,
                       // general-purpose, hoặc custom type
})
```

Khi main agent quyết định cần help, nó gọi AgentTool với task description và agent type. System tìm agent definition matching type, tạo query loop mới với system prompt của agent đó, và bắt đầu execution.

6.2.2 6 built-in agents

Agent	Tool subset	Vai trò
Explore	FileRead, Grep, Glob, Bash (read-only)	Khám phá codebase. READ-ONLY: cấm tạo file, sửa file, xóa file. Chuyên gia tìm kiếm.
Verification	Bash, FileRead, WebFetch, Agent (recursive)	Phá code, không fix. "Your job is not to confirm — it's to try to break it." Viết test vào /tmp, dọn dẹp sau.
General-purpose	['*'] (tất cả tools)	Agent đa năng. "Complete the task fully — don't gold-plate, but don't leave it half-done."
Plan	FileRead, Grep, Glob, Bash (read-only), Agent	Kiến trúc sư. READ-ONLY. Phân tích codebase và thiết kế implementation plan.
Claude Code Guide	FileRead, Grep, Glob, Bash, WebFetch, WebSearch	Chuyên gia Claude Code. Fetch docs từ code.claude.com, giúp user hiểu features.
Statusline Setup	FileRead, Bash, Config	Chuyên biệt: đọc shell config, convert PS1, viết statusLine setting.

Explore agent đáng chú ý: system prompt của nó mở đầu bằng "=== CRITICAL: READ-ONLY MODE — NO FILE MODIFICATIONS ===", gạch dưới 3 lần rằng KHÔNG ĐƯỢC sửa file. Tool subset enforce cùng constraint: không có FileWrite, FileEdit. Đây là defense-in-depth cho agents: constraint ở cả prompt level (instruction) VÀ tool level (capability). Agent không thể sửa file kể cả khi nó "quyết định" cần sửa.

Verification agent đáng chú ý hơn: system prompt ghi rõ hai failure patterns cần tránh: (1) "verification avoidance" — đọc code, tường thuật sẽ test gì, rồi ghi PASS mà không thật sự chạy; (2) "being seduced by the first 80%" — thấy UI đẹp rồi pass, không nhận ra nửa nút bấm không hoạt động. "The caller may spot-check your commands by re-running them." Đây là adversarial prompt design — system prompt đã anticipate cách agent sẽ cheat.

6.2.3 Custom agents

Ngoài 6 built-in, user có thể define custom agents trong .claude/agents/ bằng markdown files. Agent definition gồm frontmatter (tools, whenToUse, model, memory, isolation, hooks, mcpServers, permissionMode, effort, maxTurns) và markdown body (system prompt). loadAgentsDir.ts (755 dòng) xử lý discovery và loading.

Plugin agents bị hạn chế: agents từ plugins KHÔNG được set permissionMode, hooks, hoặc mcpServers ở per-agent level. Comment trong code giải thích: "Plugins are third-party marketplace code; these fields escalate what the agent can do beyond

what the user approved at install time." Muốn level control đó, phải define agent trong .claude/agents/ — nơi user explicitly viết.

* * *

6.3 Pattern B: Coordinator mode — Cấm lãnh đạo code

Coordinator mode là architectural pattern quan trọng nhất cho multi-agent systems. Ý tưởng cốt lõi: agent điều phối bị CẤM thực thi.

6.3.1 Restricted toolset

```
// src/coordinator/coordinatorMode.ts:29-34
const INTERNAL_WORKER_TOOLS = new Set([
  TEAM_CREATE_TOOL_NAME,   // Tạo team
  TEAM_DELETE_TOOL_NAME,   // Xóa team
  SEND_MESSAGE_TOOL_NAME,  // Gửi message cho agent khác
  SYNTHETIC_OUTPUT_TOOL_NAME, // Output cho user
])
```

Coordinator KHÔNG có: Bash, FileRead, FileWrite, FileEdit, Grep, Glob, Agent (spawn subagent), WebFetch, WebSearch. Nó chỉ có 4 tools: tạo team, xóa team, gửi message, và output. Muốn đọc file? Phải giao cho worker. Muốn chạy test? Phải giao cho worker. Muốn sửa code? Phải giao cho worker.

Tại sao constraint này quan trọng? Vì nếu coordinator có thể code, nó sẽ code. LLMs tend to take the shortest path. Nếu coordinator nhận task "refactor module A", và nó có FileEdit tool, nó sẽ tự sửa thay vì delegate. Kết quả: không có parallelism, không có specialization, coordinator trở thành single-agent with extra steps. Bằng cách CẤM coordinator dùng execution tools, hệ thống BUỘC nó phải chia task và delegate.

6.3.2 Communication qua SendMessage

Coordinator giao tiếp với workers qua SendMessage tool. Message types:

```
// src/tools/SendMessageTool/SendMessageTool.ts
const StructuredMessage = z.discriminatedUnion('type', [
  z.object({
    type: z.literal('shutdown_request'),
    reason: z.string().optional()
  }),
  // ... more message types
])
```

Workers nhận messages qua mailbox system (utils/teammateMailbox.ts). Message pipeline: coordinator gửi SendMessage → message enqueue vào target agent's mailbox → target agent's query loop đọc mailbox ở đầu mỗi iteration → message inject vào conversation. Asynchronous, non-blocking.

6.3.3 Team lifecycle

TeamCreateTool tạo team definition (team file trên disk), assign teammates, register cho session cleanup. TeamDeleteTool dọn dẹp. Team file chứa: team name, members, description, model config. Mỗi teammate nhận unique color (agentColorManager.ts) để UI phân biệt.

Session mode matching: matchSessionMode() trong coordinatorMode.ts đảm bảo resumed session giữ đúng mode. Nếu session gốc là coordinator mode, resume cũng phải coordinator mode. Nếu mismatch, hệ thống flip env var và trả warning. Đây ngăn bug: user start coordinator session, close terminal, resume — nếu resume ở normal mode, workers từ session cũ không nhận message vì coordinator không tồn tại.

* * *

6.4 Pattern C: Fork agent — Isolation bằng worktree

Fork là pattern phức tạp nhất: clone toàn bộ conversation context, tạo Git worktree riêng, chạy agent song song trên branch riêng, merge khi xong.

6.4.1 Fork agent definition

```
// src/tools/AgentTool/forkSubagent.ts:60-71
export const FORK_AGENT = {
  agentType: 'fork',
  tools: ['*'],              // Full tool access
  maxTurns: 200,             // Safety limit
  model: 'inherit',          // Same model as parent
  permissionMode: 'bubble',  // Bubble lên parent
  source: 'built-in',
}
```

3 design decisions quan trọng:

tools: ['*'] với useExactTools — fork child nhận chính xác tool pool của parent, không phải wildcard. Tại sao? Prompt cache sharing. Nếu fork child có different tool list, API request prefix khác parent, cache miss. Bằng cách clone exact tools, fork children share prompt cache với parent.

model: 'inherit' — fork child dùng cùng model với parent. Tại sao? Context length parity. Nếu parent dùng model có 200K context, child phải dùng cùng model để shared conversation history không bị truncate.

permissionMode: 'bubble' — khi fork child cần permission (ví dụ chạy sudo), request nổi bọt lên parent terminal. Child KHÔNG tự approve. Đây là safety measure:

user chỉ nhìn thấy permission requests ở terminal chính, không bị fragmented across multiple agent terminals.

6.4.2 Prompt cache sharing

Fork architecture optimize cho prompt cache bằng cách đảm bảo byte-identical API request prefixes giữa fork children:

```
// forkSubagent.ts comment:
// For prompt cache sharing, all fork children must
// produce byte-identical API request prefixes.
// This function:
// 1. Keeps full parent assistant message (all blocks)
// 2. Replaces ALL tool_result blocks with identical
//    placeholder text

const FORK_PLACEHOLDER_RESULT =
  'Fork started — processing in background'
```

Tất cả tool_result blocks trong shared context được replace bằng cùng placeholder text. Nghĩa là: nếu parent có 10 tool calls trong history, và spawn 3 fork children, cả 3 children gửi API request với cùng prefix (system prompt + shared history + placeholder results). API server cache prefix này, 3 children share cache. Tiết kiệm tokens và latency.

6.4.3 Anti-recursive fork

```
// forkSubagent.ts:78-80
export function isInForkChild(messages) {
  return messages.some(m =>
    m.type === 'user' &&
    m.message.content.some(b =>
      b.type === 'text' &&
      b.text.includes(`<${FORK_BOILERPLATE_TAG}>`))
  )
}
```

Fork children giữ Agent tool trong pool (cho cache sharing), nhưng nếu child cố spawn fork, isInForkChild() phát hiện boilerplate tag trong conversation history và reject. Không có guard này, fork child spawn fork child spawn fork child — exponential explosion.

6.4.4 Worktree isolation

Khi fork agent có isolation: 'worktree' (từ agent definition frontmatter), hệ thống tạo Git worktree riêng. Mỗi agent làm việc trên branch riêng, trong directory riêng. File changes không nhìn thấy giữa agents cho đến khi merge.

buildWorktreeNotice() inject vào agent's context: "isolated git worktree at ${worktreeCwd}". Agent biết mình đang ở worktree riêng, không ảnh hưởng main

branch. Sau khi agent hoàn thành, hasWorktreeChanges() kiểm tra có changes không, removeAgentWorktree() dọn dẹp.

Đây giải quyết bài toán mà hầu hết multi-agent framework bỏ qua: file conflict giữa parallel agents. Khi 2 agents cùng sửa package.json, ai thắng? Với worktree, mỗi agent sửa copy riêng. Merge conflict được xử lý bằng Git merge — tool đã proven cho conflict resolution qua 20 năm.

* * *

6.5 runAgent() — Nơi mọi pattern hội tụ

Cả 3 patterns đều đi qua cùng một function: runAgent() trong runAgent.ts. Đây là async generator (giống query loop ở Chương 3) yield messages cho parent:

```
// src/tools/AgentTool/runAgent.ts:248 (simplified)
export async function* runAgent({
  agentDefinition,    // Agent type + config
  task,               // Task description
  forkContextMessages,// Shared context (for fork)
  worktreePath,       // Isolation path (optional)
  override,           // System prompt override etc.
}) {
  // 1. Resolve model (agent-specific or inherit parent)
  const model = getAgentModel(agentDefinition, ...)

  // 2. Build context messages
  const contextMessages = forkContextMessages
    ? filterIncompleteToolCalls(forkContextMessages)
    : []  // Empty for fresh subagents

  // 3. Resolve permission mode
  const agentPermissionMode = agentDefinition.permissionMode
  // 'bubble' → show prompts in parent terminal
  // 'default' → normal permission flow

  // 4. Build system prompt
  const agentSystemPrompt = override?.systemPrompt
    || getSystemPrompt(agentDefinition, ...)

  // 5. Run query loop (same as main loop!)
  for await (const message of query({...})) {
    yield message  // Stream to parent
  }
}
```

Insight cốt lõi: subagent chạy cùng query() function với main agent. Không có "subagent query loop" riêng. Cùng while(true), cùng 4 phases, cùng concurrency partitioning, cùng permission pipeline, cùng context defense. Subagent là recursive application of the same architecture. Đây là composability ở mức kiến trúc: mọi agent, dù main hay sub hay fork, đều là instances of cùng một engine.

* * *

6.6 Agent memory — Nhớ qua sessions

Agents có thể persistent memory, độc lập với main conversation memory. agentMemory.ts + agentMemorySnapshot.ts (2 files) quản lý:

Memory scope: 3 levels — 'user' (global, shared across projects), 'project' (per-project directory), 'local' (per-session). Agent definition declare memory scope trong frontmatter: memory: 'project' nghĩa là agent nhớ information specific to this project.

Memory injection: Khi agent có memory scope, loadAgentMemoryPrompt() append memory context vào system prompt. Agent cũng tự động nhận FileWrite/FileEdit/FileRead tools (nếu chưa có) để đọc/ghi memory files.

Memory snapshots: agentMemorySnapshot.ts cho phép agent save/restore state snapshots. checkAgentMemorySnapshot() kiểm tra snapshot tồn tại, initializeFromSnapshot() restore. Pattern này cho phép "agent sleep and wake" — agent hoàn thành task, save snapshot, session kết thúc. Session mới, agent resume từ snapshot.

* * *

6.7 Kết nối với Vibecode v7.0

Ba patterns multi-agent của Claude Code map trực tiếp vào Vibecode methodology:

Claude Code	Cơ chế	Vibecode v7.0 tương đương
Coordinator restricted toolset	4 tools only. Cấm code bằng code.	FORBIDDEN action list. Chủ thầu bị cấm code bằng protocol.
Explore agent read-only	Prompt + tool subset đều enforce read-only.	SCAN step: Thợ chỉ đọc, không sửa. Defense-in-depth.
Verification agent adversarial	"Try to break it, not confirm it works."	VERIFY step: Chủ thầu kiểm tra kỹ. QA Protocol Tier 1-3.
Fork worktree isolation	Git worktree per agent. Merge sau.	Multi-builder: EXCLUSIVE TIPs không overlap files.
permissionMode: bubble	Permission nổi bọt lên parent.	Escalation Protocol: Level 2 (Thợ → Chủ thầu).

Khác biệt cốt lõi: Claude Code enforce bằng code (restricted tool list, tool subset). Vibecode enforce bằng protocol (FORBIDDEN action list, Completion Report

review). Cùng triết lý, khác cơ chế. Cả hai đều cần: code enforcement cho runtime safety, protocol enforcement cho human oversight.

* * *

6.8 Bài học rút ra — 2 Patterns

Pattern #7: Coordinator restriction

Agent điều phối bị CẤM dùng execution tools. Chỉ có coordination tools (create team, send message, output). Constraint bằng code, không bằng instruction. Buộc coordinator phải delegate thay vì tự làm. Kết quả: parallelism tự nhiên, specialization tự nhiên.

Áp dụng: Manager agent trong enterprise workflow. Project coordinator trong multi-team development. Orchestrator trong data pipeline. Bất kỳ system nào cần separation of planning and execution.

Pattern #8: Fork isolation via worktree

Khi agents cần sửa code song song, mỗi agent nhận workspace riêng (Git worktree). Permission bubble lên parent. Context shared nhưng workspace isolated. Merge bằng Git. Anti-recursive guard ngăn fork explosion.

Áp dụng: Parallel development: N developers/agents work on N features simultaneously. Simulation: N units operate in N environments, merge results. Testing: N test suites run in N sandboxes. Bất kỳ system nào cần parallel mutation of shared state.

* * *

Chương tiếp theo sẽ đi vào Background Tasks — 7 loại task chạy song song, lifecycle 5 trạng thái, và cơ chế disk-based output cho phép resume sau crash. Đây là layer cho phép agents chạy independent mà coordinator vẫn giám sát được.

CHƯƠNG 7

Background Tasks

7 loại task, 5 trạng thái, disk-based output

> *"Khi agent chỉ chạy foreground, bạn đợi. Khi agent chạy background, bạn tiếp tục làm việc. Đó là sự khác biệt giữa tool và assistant."*

7.1 Tại sao cần background tasks

Ở Chương 6, ta thấy agents có thể spawn agents khác. Nhưng trong pattern A (subagent), parent agent đợi child hoàn thành. Đây là blocking: parent không làm gì khác trong khi child chạy. Background tasks giải quyết vấn đề này: child chạy independent, parent tiếp tục. Kết quả giao tiếp qua file và notifications.

Claude Code có 3.286 dòng code cho task system (12 files trong src/tasks/). Hệ thống hỗ trợ 7 loại task với lifecycle thống nhất.

* * *

7.2 7 loại task

Task type	Mô tả	Use case
local_bash	Shell process chạy nền. Monitor output file.	npm test, build commands, long-running dev servers
local_agent	Agent chạy nền với own query loop.	Background research, code review, parallel refactoring
remote_agent	Agent chạy trên remote server (teleport).	Heavy computation, remote environment access
in_process_teammate	Teammate trong coordinator mode.	Team workers trong swarm coordination
local_workflow	Scripted workflow execution.	Automated multi-step pipelines (feature-gated)
monitor_mcp	MCP server health monitor.	Keep MCP connections alive, detect failures
dream	Memory consolidation agent. Chạy giữa sessions.	Auto-organize memories, prune stale data, summarize sessions

Dream task đặc biệt thú vị: đây là agent "mơ" giữa sessions. Nó chạy background, review sessions gần đây, consolidate memories, prune data stale. DreamTask.ts mô tả 4 phases (orient/gather/consolidate/prune) nhưng không enforce — agent tự quyết. Biến "idle time" thành "improvement time."

* * *

7.3 Lifecycle: 5 trạng thái

```
pending → running → completed
                  → failed
                  → killed
```

pending: task được tạo nhưng chưa bắt đầu. running: đang thực thi. completed: hoàn thành thành công. failed: thất bại (error). killed: bị user hoặc system dừng. isTerminalTaskStatus() trả true cho completed/failed/killed — task không bao giờ chuyển ngược.

Guard quan trọng: "Used to guard against injecting messages into dead teammates, evicting finished tasks from AppState, and orphan-cleanup paths." (comment từ Task.ts:21). Không có guard này, system có thể gửi message cho agent đã chết — message mất, không ai xử lý.

* * *

7.4 Disk-based output — outputFile + outputOffset

Task output không lưu trong memory. Nó được ghi ra file trên disk:

```
// src/Task.ts:54-55
outputFile: string   // Path tới output file
outputOffset: number // Đọc từ vị trí nào (incremental)
```

outputOffset cho phép incremental reading: coordinator đọc output từ offset 0 đến 1000, lần sau đọc từ 1000 đến 2500. Không cần đọc lại từ đầu. Pattern này cho phép: (1) Resume sau crash — file trên disk survive process restart. (2) Size không giới hạn — output dài không tốn memory. (3) Multiple readers — coordinator và UI đều đọc cùng file, không race condition.

Stall detection cho shell tasks: LocalShellTask.tsx kiểm tra output file mỗi 5 giây (STALL_CHECK_INTERVAL_MS). Nếu không có output mới sau 45 giây (STALL_THRESHOLD_MS), kiểm tra cuối file có giống interactive prompt không ("Y/n?", "Continue?", "Press any key"). Nếu có → gửi notification cho agent: command đang chờ input. Agent có thể quyết định gửi input hoặc kill task.

* * *

7.5 Task notifications — Giao tiếp async

Khi background task hoàn thành (hoặc fail), nó enqueue notification:

```
// Notification format (XML)
<task-notification>
  <task-id>abc123</task-id>
  <status>completed</status>
  <output-file>/path/to/output</output-file>
  <summary>Tests passed: 47/47</summary>
</task-notification>
```

Notification inject vào main agent's query loop ở đầu iteration tiếp (Phase 1 context assembly). Agent nhìn thấy notification như một message, quyết định hành động tiếp: đọc full output, spawn follow-up task, hoặc báo user.

Notification delivery tuân thủ agent scoping: main thread chỉ nhận notifications cho agentId === undefined. Subagents chỉ nhận notifications cho agentId của mình. Không cross-agent pollution.

* * *

7.6 Kết nối với Query Loop

Background tasks inject vào query loop qua 2 điểm:

1. Task output as attachment: generateTaskAttachments() đọc output file từ outputOffset, tạo attachment message, inject vào toolResults. Agent nhìn thấy output mới từ background tasks mà không cần explicitly poll.

2. Queued commands drain: getCommandsByMaxPriority() drain pending notifications. Nếu Sleep tool vừa chạy (sleepRan), drain cả 'later' priority. Nếu không, chỉ drain 'next' priority. Pattern: Sleep = "tôi đang đợi, cho tôi thấy mọi thứ." No Sleep = "tôi đang bận, chỉ cho tôi urgent."

* * *

7.7 Bài học rút ra

Pattern: Disk-based async output. Tasks ghi output ra file, readers đọc incremental qua offset. Resume-safe, size-unlimited, multi-reader. Notifications inject vào consumer's event loop. Stall detection cho long-running processes. Agent scoping ngăn cross-agent pollution.

Áp dụng: Multi-agent simulations: units report qua file, coordinator reads incremental. CI/CD: build output streamed, monitor reads tail. IoT: sensor data written to file, multiple consumers read at own pace.

* * *

Chương tiếp theo sẽ đi vào Context Defense — 5 lớp chống mất trí nhớ. Khi agents chạy hàng giờ với background tasks, context window đầy. Làm sao giữ conversation sống?

CHƯƠNG 8

Context Defense

5 lớp chống mất trí nhớ

> *"LLM không có bộ nhớ thật. Nó có context window — một cửa sổ hữu hạn nhìn vào cuộc hội thoại. Khi cửa sổ đầy, agent quên. Quản lý context không phải optimization, nó là survival."*

8.1 Bài toán: context window hữu hạn

Mọi LLM có context window giới hạn: 128K, 200K, thậm chí 1M tokens. Nghe có vẻ nhiều, nhưng trong agentic session, context bị tiêu thụ nhanh chóng. Mỗi tool call tạo tool_use block (input) + tool_result block (output). Một Bash command trả 500 dòng output ≈ 2.000 tokens. 50 tool calls = 100.000 tokens chỉ cho tool results. Thêm system prompt, conversation history, thinking blocks — context window đầy sau 30-60 phút agentic work.

Khi context đầy, hai điều xảy ra: (1) API trả prompt_too_long error — request bị reject; (2) hoặc tệ hơn, thông tin quan trọng ở đầu conversation bị "đẩy ra" khỏi attention window — agent "quên" context ban đầu. Claude Code invest 3.971 dòng code (13 files) để giải quyết bài toán này.

* * *

8.2 Nguyên tắc thiết kế: escalating defense

Mỗi lớp defense đắt hơn lớp trước. Chạy lớp rẻ nhất trước. Chỉ escalate khi lớp trước không đủ. Mỗi lớp chỉ chạy một lần per iteration. Comment trong query.ts:1066 ghi rõ: "collapse drain first (cheap, keeps granular context), then reactive compact (full summary). Single-shot on each." Không retry cùng strategy.

* * *

8.3 Lớp 1: Tool result truncation

Rẻ nhất. Mỗi tool có maxResultSizeChars. Khi output vượt limit, content bị persist ra disk, chỉ giữ pointer trong conversation. applyToolResultBudget() trong utils/toolResultStorage.ts xử lý. ContentReplacementState track những gì đã được

persist. Khi agent cần đọc lại, nó dùng FileRead tool — lazy loading thay vì eager loading.

* * *

8.4 Lớp 2: Microcompact

530 dòng (microCompact.ts). Loại bỏ tool results cũ tại compact boundaries. Không cần gọi LLM. Hoạt động dựa trên tool_use_id: đánh dấu tool results đã "stale", replace bằng summary ngắn hoặc xóa hẳn. Tạo microcompact boundary message. Rẻ hơn LLM summarize nhưng mất granularity.

Variant: cachedMicrocompact (feature-gated, 7 dòng stub) — dùng API cache editing để xóa tool results mà không tái tạo conversation. Metric tracked: cache_deleted_input_tokens từ API response.

* * *

8.5 Lớp 3: Auto-compact

Lớp chính. 2.056 dòng (autoCompact.ts + compact.ts). Khi token count vượt effectiveContextWindow, gọi LLM summarize toàn bộ conversation:

```
// autoCompact.ts:32-40
function getEffectiveContextWindowSize(model) {
  const reserved = Math.min(
    getMaxOutputTokensForModel(model),
    20_000  // p99.99 compact output = 17,387 tokens
  )
  return getContextWindowForModel(model) - reserved
}
```

20.000 tokens reserved cho compact summary output. Số này dựa trên data thống kê: p99.99 = 17.387 tokens. Đây là engineering decision dựa trên empirical data, không phải lý thuyết.

Compact boundary message: sau khi LLM summarize, hệ thống tạo SystemCompactBoundaryMessage. Mọi messages trước boundary bị loại khỏi API request. Chỉ giữ summary + messages sau boundary. Đây là "quên có kiểm soát" — hệ thống quyết định quên gì, giữ gì, thay vì để context tự trôi.

Compact cũng trigger memory extraction: session memory compaction (sessionMemoryCompact.ts, 630 dòng) chạy cùng lúc. Nó extract key facts từ conversation đang bị compact, lưu vào session memory. Facts này sẽ được attach ở session tiếp — "quên conversation, nhớ lessons."

* * *

8.6 Lớp 4: Reactive compact

Emergency. Trigger khi nhận prompt_too_long (HTTP 413) error giữa turn. Khác auto-compact (proactive, chạy đầu iteration), reactive compact chạy sau khi API đã reject request:

```
// query.ts:1066-1069 (comment)
// collapse drain first (cheap, keeps granular context),
// then reactive compact (full summary). Single-shot on
// each — if a retry still 413's, the next stage handles
// it or the error surfaces.
```

Flow: API trả 413 → error bị withheld (không yield cho UI) → thử context collapse drain → nếu không đủ → thử reactive compact (LLM summarize) → retry API call → nếu vẫn 413 → surface error cho user.

hasAttemptedReactiveCompact guard: set true sau lần đầu. Ngăn infinite loop: compact → vẫn quá lớn → compact lại → vẫn quá lớn → ... Circuit breaker. Nếu reactive compact đã chạy và vẫn fail, hệ thống kết luận: conversation fundamentally quá lớn, cần user intervention.

* * *

8.7 Lớp 5: Context collapse

Stubbed trong bản leak (6 dòng). Nhưng query.ts reference nó extensively. Dựa trên code comments, context collapse là read-time projection: không thay đổi messages trong memory, chỉ thay đổi cách view messages khi gửi API. "Nothing is yielded — the collapsed view is a read-time projection." (query.ts:433)

Collapse drain: khi 413 error xảy ra, hệ thống drain staged collapses (recoverFromOverflow) trước khi escalate lên reactive compact. Rẻ hơn compact vì không cần LLM call.

* * *

8.8 Memory system — Nhớ qua sessions

Context defense giữ conversation sống trong session. Memory system giữ knowledge qua sessions. 8 files trong src/memdir/:

File	Vai trò
extractMemories.ts	Chạy cuối session. Forked agent extract key facts từ

	conversation.
memoryScan.ts	Scan memory directory, index available memories.
findRelevantMemories.ts	Đầu session mới, tìm memories relevant cho current context.
memdir.ts	Core: loadMemoryPrompt() inject memories vào system prompt.
memoryAge.ts	Memory decay: memories cũ giảm relevance score.
teamMemPaths/Prompts.ts	Team memory: shared memory giữa agents trong cùng team.

Lifecycle hoàn chỉnh: Session chạy → tool calls, conversations → session kết thúc → extractMemories (background, không block) → Session mới bắt đầu → memoryScan + findRelevantMemories → loadMemoryPrompt inject vào system prompt → Agent "nhớ" relevant facts từ sessions trước.

Team memory (teamMemPaths.ts, teamMemPrompts.ts): khi nhiều agents trong cùng team (coordinator mode), chúng share memory space. Agent A extract memory, Agent B đọc được ở session sau. Shared organizational knowledge.

* * *

8.9 Bài học rút ra — Pattern #9

Pattern #9: Multi-layer context defense

5+ lớp escalating: truncate (gần zero cost) → microcompact (no LLM) → auto-compact (1 LLM call) → reactive compact (emergency LLM call + retry) → context collapse (read-time projection). Mỗi lớp chỉ chạy 1 lần. Circuit breaker (hasAttemptedReactiveCompact) ngăn infinite loop. Compact boundary message = "quên có kiểm soát." Memory system = "quên conversation, nhớ lessons."

Áp dụng: Bất kỳ long-running AI session nào. Enterprise chatbot handling multi-hour customer interactions. Simulation engines running extended scenarios. Research agents processing large document sets. Vibecode v7.0 Context Defense 5 layers (CONTEXT CARRY → CONTEXT FORWARD → Milestone Summary → Session Handover → X-Ray Archive).

* * *

Phần III kết thúc tại đây. Ba chương vừa rồi đã giải phẫu cách hệ thống scale từ một agent lên nhiều agents: spawn và phối hợp (Chương 6), chạy nền (Chương 7),

giữ trí nhớ (Chương 8). Phần IV sẽ mở rộng ra ecosystem: Skills, Plugins, UI engine, và cách kết nối tất cả vào phương pháp luận thực tiễn.

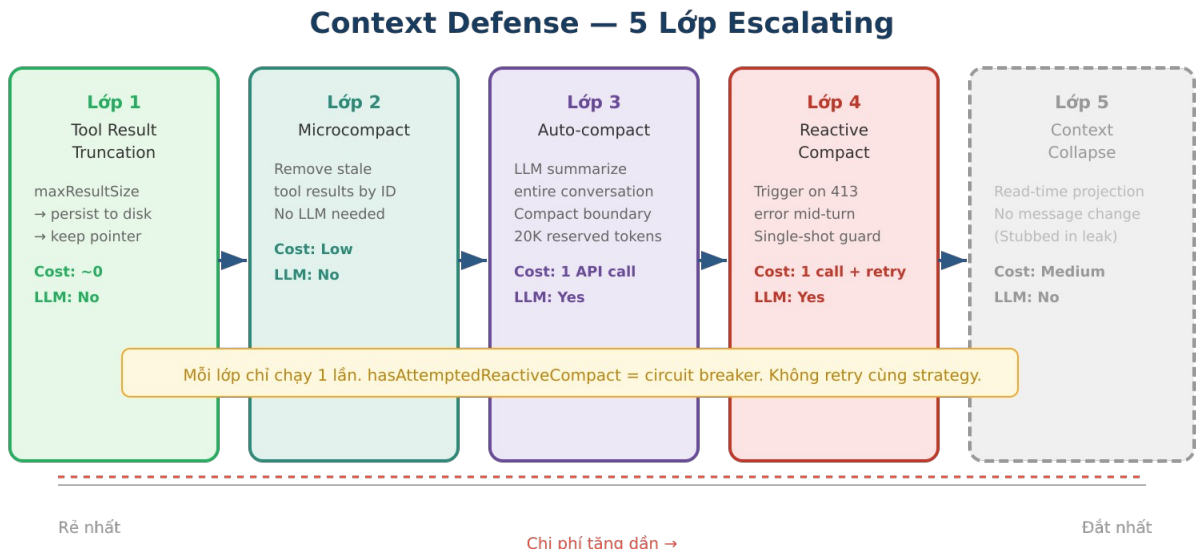
PHẦN IV — ECOSYSTEM

Mở rộng và phương pháp luận

* * *

Hình 8.1: Context Defense — 5 Lớp escalating

(Chương 8)



Từ rẻ nhất (Tool Result Truncation, cost ~0) đến đắt nhất (Context Collapse). Mỗi lớp chỉ chạy 1 lần. hasAttemptedReactiveCompact là circuit breaker.

CHƯƠNG 9

Skill System — Programmable Prompts

> *"Tool thay đổi khả năng của agent. Skill thay đổi chỉ dẫn của agent. Một cái cho agent tay chân mới. Cái kia cho agent kiến thức mới."*

9.1 Skill khác Tool thế nào

Trong 43 tools của Claude Code, mỗi tool thực thi một hành động: đọc file, chạy bash, tìm kiếm web. Tool thay đổi capability — agent có thể làm gì. Skill khác: nó là markdown template được inject vào conversation như user message. Skill thay đổi instruction — agent biết gì.

Ví dụ: khi user gõ /verify, hệ thống không chạy verification code. Nó inject verification prompt vào conversation: "You are a verification specialist. Your job is not to confirm — it's to try to break it." Agent nhận prompt này và tự thực hiện verification bằng tools có sẵn. Skill không thêm tool mới, nó thay đổi cách agent dùng tools đã có.

* * *

9.2 5 nguồn skill

Nguồn	Vị trí	Ai tạo	Số lượng trong leak
Bundled	Compiled vào CLI binary	Anthropic engineering team	16 (11 always-on, 5 gated)
User	~/.claude/skills/	User cá nhân	Không giới hạn
Project	.claude/skills/	Team dự án	Không giới hạn
MCP	Remote MCP servers	MCP server providers	Depends on servers
Managed	Enterprise policy path	Enterprise admin	Policy-controlled

* * *

9.3 SKILL.md anatomy

Mỗi skill là một thư mục chứa SKILL.md. File này gồm frontmatter (YAML metadata) và markdown body (prompt template):

```
---
name: api-endpoint-scaffold
description: Scaffold new REST API endpoint
when_to_use: When creating new API endpoints
paths: ["**/routes/**", "**/api/**"]
allowed-tools: [Read, Write, Edit, Bash]
arguments: [endpoint_name, http_method]
context: fork  # or inline (default)
agent: general-purpose  # optional delegation
effort: high
hooks:
  PostToolUse:
    - match_tool: Write
      command: npm run lint ${file_path}
---

# Skill: API Endpoint Scaffold

## Context
Read existing routes: !`find . -name "*.py" -path "*/routes/*"`

## Task
Create endpoint ${endpoint_name} with method ${http_method}...
```

Frontmatter fields quan trọng: name (unique identifier), description (shown to LLM for selection), when_to_use (LLM dùng để quyết định khi nào invoke), paths (gitignore patterns cho conditional activation), allowed-tools (restrict tool access khi skill chạy), arguments (template variables), context (inline inject vào conversation hoặc fork spawn subagent), hooks (lifecycle events khi skill chạy).

* * *

9.4 3 discovery modes

9.4.1 Static (startup)

getSkillDirCommands() chạy lúc startup, memoized. Scan 4 sources song song: managed, user, project, additional dirs. Deduplicate by resolved path (realpath, xử lý symlinks). Skills load một lần, cache cho toàn session.

9.4.2 Dynamic (file walk)

discoverSkillDirsForPaths(): khi agent touch file (FileRead/Write/Edit), system walk up directory tree tìm .claude/skills/ ở mọi ancestor directory. Deeper = higher priority. Pattern này cho phép monorepo: packages/auth/.claude/skills/ chứa auth-specific skills, packages/api/.claude/skills/ chứa api-specific skills. Khi agent mở file trong packages/auth/, auth skills tự load mà không cần config.

Gitignore check: trước khi load skill directory, system check isPathGitignored(). Skills trong node_modules/ hoặc .gitignore-d directories bị bỏ qua. Ngăn supply-chain injection qua npm packages chứa .claude/skills/.

9.4.3 Conditional (paths activation)

Skills có paths frontmatter chỉ activate khi file matching pattern được touch. activateConditionalSkillsForPaths(): dùng ignore library (gitignore-style matching). Skill database-migration với paths: ["**/migrations/**"] chỉ xuất hiện khi agent mở file trong migrations/. Giảm noise, tăng relevance.

* * *

9.5 Shell-in-prompt — Dynamic context injection

Skills có thể chứa inline shell commands trong markdown body:

```
## Context
Read existing routes: !`find . -name "*.py" -path "*/routes/*" | head -10`
Current branch: !`git branch --show-current`
```

Khi skill được invoke, executeShellCommandsInPrompt() chạy shell commands trước, replace output vào prompt. Agent nhận prompt với context thật: danh sách routes hiện tại, branch hiện tại. Dynamic context mà không cần agent tự chạy commands.

Security: MCP skills bị block shell execution. Comment trong code: "MCP skills are remote and untrusted — never execute inline shell commands from their markdown body." Chỉ local và user-created skills được chạy shell. Ngăn remote code execution qua MCP skill injection.

* * *

9.6 TIP → Skill pipeline (Vibecode v7.0)

Vibecode v6.0 TIPs là one-shot. v7.0 thêm promotion path: TIP pattern dùng 3+ lần → convert thành Skill. Skill tự activate by file path. Knowledge compounds thay vì evaporates.

Áp dụng: MRP-specific TIPs về BOM validation → BOM validation skill. Auth integration TIPs → auth integration skill. Database migration TIPs → migration skill. Mỗi skill tự activate khi touch relevant files. Chủ thầu không cần lặp lại instructions cho patterns đã proven.

* * *

Chương tiếp theo sẽ phân tích Plugin System — 31.484 dòng code, marketplace ecosystem, 4 extension points, 26 lifecycle hooks. Đây là blueprint cho RTR app ecosystem.

CHƯƠNG 10

Plugin System — Marketplace và 4 extension points

> *"Một hệ thống không có extension mechanism sẽ chết khi nó không còn đủ feature. Một hệ thống có extension mechanism chỉ chết khi không còn ai muốn extend."*

10.1 Tại sao cần plugin

Claude Code có 43 built-in tools, 88 slash commands, 16 bundled skills. Nhưng mỗi team, mỗi project, mỗi workflow có nhu cầu riêng. Không thể build-in mọi thứ. Plugin system (31.484 dòng, 65+ files) cho phép third-party extend Claude Code mà không fork codebase.

* * *

10.2 4 extension points

Extension	Cung cấp	Ví dụ
Commands	Slash commands từ markdown files. Tương tự skills nhưng scoped cho plugin.	feature-dev plugin: /feature-dev command
Agents	Custom AI agents với own system prompts, tool subsets, memory scopes.	feature-dev: code-architect, code-explorer, code-reviewer agents
Hooks	26 lifecycle events. Shell commands chạy khi event fire.	PostToolUse hook: auto-lint sau mỗi FileWrite
Servers	MCP servers và LSP servers. External tool providers.	typescript-lsp plugin: TypeScript language server

Plugin feature-dev minh họa sức mạnh: một install cung cấp 3 agents (code-architect lập kế hoạch, code-explorer khám phá codebase, code-reviewer review changes) + 1 command (/feature-dev). Toàn bộ feature development workflow trong một plugin.

* * *

10.3 26 lifecycle hook events

Hooks là extension point mạnh nhất. 26 events bao phủ toàn bộ lifecycle:

Category	Events
Tool lifecycle	PreToolUse, PostToolUse, PostToolUseFailure
Permission	PermissionDenied, PermissionRequest
Session	SessionStart, SessionEnd, Setup
Agent lifecycle	SubagentStart, SubagentStop, TeammateIdle
Task	TaskCreated, TaskCompleted, Stop, StopFailure
Context	PreCompact, PostCompact, Notification
Filesystem	FileChanged, CwdChanged, WorktreeCreate, WorktreeRemove
UI / Config	Elicitation, ElicitationResult, ConfigChange, InstructionsLoaded, UserPromptSubmit

PreToolUse + PostToolUse là cặp hook quan trọng nhất. PreToolUse có thể block tool execution (return deny). PostToolUse có thể trigger follow-up actions. Ví dụ: hook auto-lint chạy eslint sau mỗi FileWrite, inject kết quả lint vào conversation. Agent tự sửa lint errors mà user không cần nhắc.

* * *

10.4 Security sandbox cho plugins

Plugin là third-party code. Claude Code enforce security boundaries:

1. Plugin agents CANNOT set permissionMode, hooks, mcpServers per-agent. Comment: "these fields escalate what the agent can do beyond what the user approved at install time." Plugin-level hooks/MCP OK (install-time trust), per-agent escalation NOT OK.

2. Enterprise policies: blockedMarketplaces (blacklist), strictKnownMarketplaces (whitelist). marketplace-blocked-by-policy error cho unauthorized sources.

3. Dependency resolver: plugins có thể declare dependencies. dependency-unsatisfied error nếu required plugin chưa installed hoặc disabled.

* * *

10.5 Reconciliation-based install

Plugin install không phải "download then done." Nó dùng Kubernetes-style reconciliation:

```
// PluginInstallationManager.ts (simplified)
const declared = getDeclaredMarketplaces()
const materialized = loadKnownMarketplacesConfig()
const diff = diffMarketplaces(declared, materialized)
// diff.missing: cần install
// diff.sourceChanged: cần update
await reconcileMarketplaces(diff) // Apply changes
```

diffMarketplaces() tính delta giữa desired state (settings) và actual state (disk). reconcileMarketplaces() apply changes. Background, không block startup. Hot reload qua /reload-plugins.

* * *

10.6 32 official plugins

Marketplace chính thức (extracted từ GCS bucket) chứa 32 plugins trong 4 categories:

Category	Plugins	Chức năng
LSP (12)	typescript, rust-analyzer, pyright, gopls, clangd, ruby, swift, php, kotlin, lua, jdtls, csharp	Language Server Protocol cho 12 ngôn ngữ. Type checking, diagnostics, go-to-definition.
Workflow (10)	feature-dev, code-review, pr-review-toolkit, commit-commands, code-simplifier, security-guidance, math-olympiad, playground, ralph-loop, hookify	Development workflows: feature dev, code review, PR management, security audit.
Meta (6)	plugin-dev, skill-creator, mcp-server-dev, agent-sdk-dev, claude-code-setup, claude-md-management	Tools cho building more tools: tạo plugins, skills, MCP servers.
Output (4)	explanatory-output-style, learning-output-style, example-plugin, frontend-design	Output style customization và examples.

Áp dụng cho RTR: mỗi RTR app (MRP, HRM, CRM...) có thể trở thành plugin cung cấp agents riêng (MRP BOM checker, HRM leave approval), commands riêng, hooks riêng. Auth Gateway là platform, mỗi app là plugin. Apps giao tiếp qua hook events, không hardcode integration. O(n) thay vì O(n^2).

* * *

Chương tiếp theo sẽ phân tích Terminal UI — custom Ink fork, Yoga layout engine port, và cách React reconciler render mọi thứ lên terminal.

CHƯƠNG 11

Terminal UI — React beyond the browser

> *"Terminal không phải nơi bạn chỉ in text. Nó là canvas. React reconciler cho bạn paint bất kỳ thứ gì lên bất kỳ canvas nào."*

11.1 Stack: React → Yoga → ANSI

Claude Code terminal UI là 6-layer rendering pipeline tổng cộng ~105.000 dòng:

#	Layer	Implementation	LOC
1	**React components**	390 components (REPL, permissions, diff viewer...)	81.551
2	**Ink primitives**	Box, Text, ScrollBox, Button, Link, RawAnsi	19.865 (custom fork)
3	**React reconciler**	react-reconciler → DOM tree (createNode, setAttribute)	Included in Ink
4	**Yoga layout**	Flexbox engine — pure TypeScript port from Meta's C++	2.578
5	**Event system**	Hit testing, keyboard, focus, click on terminal chars	Included in Ink
6	**Terminal I/O**	ANSI tokenizer/parser, CSI/SGR/OSC/ESC, bidi text	Included in Ink

* * *

11.2 Custom Ink fork

Claude Code không dùng Ink gốc (npm ink). Họ fork và tùy biến sâu. 19.865 dòng cho Ink layer bao gồm: custom reconciler, custom event system với hit testing, custom scroll handling, bidirectional text support, và 13 custom React hooks (useInput, useStdin, useSelection, useSearchHighlight, useTerminalViewport, useTerminalFocus...).

Tại sao fork thay vì contribute upstream? Vì mức tùy biến quá sâu: click event trên terminal characters (hit testing bằng character coordinates), custom focus

management cho permission dialogs, scroll virtualization cho output dài. Những changes này quá specific cho agentic CLI, upstream unlikely accept.

* * *

11.3 Yoga layout engine — Pure TypeScript port

Meta's Yoga là flexbox layout engine viết bằng C++, dùng trong React Native. Claude Code port nó sang 2.578 dòng TypeScript thuần (src/native-ts/yoga-layout/index.ts). Không native binary, không node-gyp, chạy trên mọi platform Bun hỗ trợ.

Pattern #17: Native module replacement. Khi native dependency gây pain (cross-platform builds, binary distribution, CI complexity), viết lại bằng TypeScript/JavaScript nếu performance budget cho phép. 2.578 dòng TS chạy chậm hơn C++ Yoga, nhưng cho terminal layout, tốc độ không phải bottleneck — network latency và LLM inference time lớn hơn nhiều bậc.

Tương tự, color-diff (Rust → TS) và file-index (fuzzy search, native → TS) cũng được port. Tổng native-ts: 4.081 dòng thay thế 3 native binaries.

* * *

11.4 390 components cho giám sát

Tại sao terminal app cần 390 React components? Vì agentic system cần giao diện giám sát:

Permission dialogs: khi agent cần approval, UI hiển thị command, context, risk level. User approve/deny/always-allow. Nhiều dialog types cho Bash, FileEdit, MCP, Agent spawning.

Agent status: khi multiple agents chạy, UI hiển thị mỗi agent với color riêng, progress indicator, current tool. CoordinatorAgentStatus.tsx, AgentProgressLine.tsx.

Diff viewer: StructuredDiff/ directory. Syntax-highlighted diffs trong terminal. Scrollable. Collapsible.

Cost tracking: CostThresholdDialog.tsx. Khi cost vượt threshold, prompt user tiếp tục hay dừng.

Context visualization: ContextVisualization.tsx. Hiển thị context window usage, token count, compact state.

* * *

11.5 Bài học rút ra

Pattern #17: Pure-TS native replacement. Port native modules sang TypeScript khi cross-platform > performance. Yoga C++ → 2.578 dòng TS. Quyết định dựa trên bottleneck analysis: nếu module không phải bottleneck, TS port là valid trade-off.

Pattern #18: Terminal-as-browser rendering. React reconciler API render bất kỳ target nào. Pipeline: React components → virtual DOM → layout (Yoga flexbox) → paint (ANSI escape sequences). Giống hệt browser rendering nhưng output ANSI thay vì pixels. Applicable cho: custom rendering targets, embedded displays, headless UIs.

* * *

Chương tiếp theo sẽ phân tích infrastructure ẩn: feature flags, multi-provider API client, và startup optimization.

CHƯƠNG 12

Infrastructure ẩn

> *"Infrastructure tốt nhất là infrastructure bạn không nhận ra đang chạy."*

12.1 Feature flags — Compile-time dead code elimination

Claude Code dùng ~20 feature flags qua bun:bundle feature() API. Khác biệt cốt lõi so với runtime feature flags (LaunchDarkly, GrowthBook): feature() quyết định lúc compile. Code path bị tắt bị loại khỏi binary hoàn toàn (dead code elimination). Binary size nhỏ hơn, startup nhanh hơn, không runtime overhead.

Flag	Ẩn feature
COORDINATOR_MODE	Multi-agent coordinator pattern (Chương 6)
FORK_SUBAGENT	Git worktree fork agents (Chương 6)
KAIROS / KAIROS_DREAM	Session transcript + Dream task (Chương 7)
REACTIVE_COMPACT	Emergency mid-turn compaction (Chương 8)
CONTEXT_COLLAPSE	Read-time context projection (Chương 8)
VOICE_MODE	Voice input/output (speech-to-text)
TRANSCRIPT_CLASSIFIER	Auto mode LLM classifier (Chương 5)
WORKFLOW_SCRIPTS	Scripted workflow execution (Chương 7)
AGENT_TRIGGERS	Cron scheduling + remote agent triggers

Trong bản leak, tất cả flags trả false (shim). Nghĩa là ~30% capabilities bị ẩn. Bản internal build có toàn bộ features bật.

* * *

12.2 Multi-provider API client

Claude Code không chỉ gọi Anthropic API. services/api/client.ts hỗ trợ 4 providers:

1. Direct Anthropic API — ANTHROPIC_API_KEY. Default path.

2. AWS Bedrock — @anthropic-ai/bedrock-sdk. AWS credentials via standard chain. Region configurable.

3. Google Vertex AI — @anthropic-ai/vertex-sdk. google-auth-library. Region per model.

4. Anthropic Foundry — @anthropic-ai/foundry-sdk. Enterprise deployment.

getAPIProvider() detect provider từ env/config. Một codebase, compile-time switch, runtime provider selection. Pattern cho multi-cloud AI deployment.

* * *

12.3 Startup optimization

Claude Code optimize startup performance bằng parallel prefetch pattern:

1. MDM prefetch: startMdmRawRead() fires subprocesses (plutil/reg query) ngay đầu main.tsx, chạy song song với module imports (~135ms). Kết quả sẵn sàng khi cần.

2. Keychain prefetch: startKeychainPrefetch() fires macOS keychain reads (OAuth + legacy API key) song song. Thay vì sequential reads (~65ms each), both complete in ~65ms total.

3. GrowthBook prefetch: Feature flag values prefetched cùng lúc auth check. Không block main thread.

4. MCP prefetch: prefetchOfficialMcpUrls() load official MCP server registry trước khi user cần.

Pattern: identify independent async operations during startup, fire all simultaneously, await results only when needed. Startup profiler (utils/startupProfiler.ts) measures each checkpoint.

* * *

12.4 Telemetry architecture

OpenTelemetry cho tracing, metrics, logs. GrowthBook cho feature flags + A/B testing. Analytics events via logEvent(). Diagnostic tracking (diagnosticTracking.ts) cho debugging. Tất cả disable-able via DISABLE_TELEMETRY=1.

Áp dụng: Bất kỳ production CLI tool nào cần observability. Pattern: OpenTelemetry for system metrics + custom analytics for product events + feature flags for gradual rollout. Disable switch for privacy.

* * *

Phần IV tiếp tục với Chương 13 (Vibecode meets Claude Code), Chương 14 (5 Blueprints ứng dụng), Chương 15 (Triết lý human-in-the-loop), và Chương 16 (Tương lai).

PHẦN V — SYNTHESIS

Kết nối và ứng dụng

* * *

CHƯƠNG 13

Vibecode gặp Claude Code

> *"Constitution cần operating system để enforce. Operating system cần constitution để biết enforce cái gì. Hai hệ thống bổ khuyết, không cạnh tranh."*

13.1 Hai hệ thống, hai tầng giải

Vibecode v6.0 và Claude Code giải cùng bài toán: làm sao để AI agents phối hợp xây phần mềm. Nhưng chúng giải ở hai tầng khác nhau. Vibecode là methodology — quy trình cho con người điều phối AI. Claude Code là runtime engine — cơ chế máy cưỡng chế quy tắc. Một cái nói "Chủ thầu không nên code." Cái kia nói "Coordinator không có Bash tool."

Sau khi phân tích 513.000 dòng code và so sánh 25 khía cạnh, kết quả matching tổng thể: 29%. Vibecode vượt trội ở QA (93%). Claude Code vượt trội ở runtime enforcement (9-18%). Cả hai đều cần cái mà bên kia có.

* * *

13.2 Bảng matching 25 khía cạnh

Kết quả đánh giá theo 7 nhóm:

Nhóm	Số khía cạnh	Match %	Ai dẫn
Vai trò và phân quyền	3	**47%**	Gần ngang
Workflow pipeline	5	**40%**	Claude Code (loop, feedback)
Task orchestration	5	**18%**	Claude Code (concurrency)
Multi-agent	4	**9%**	Claude Code (spawn,

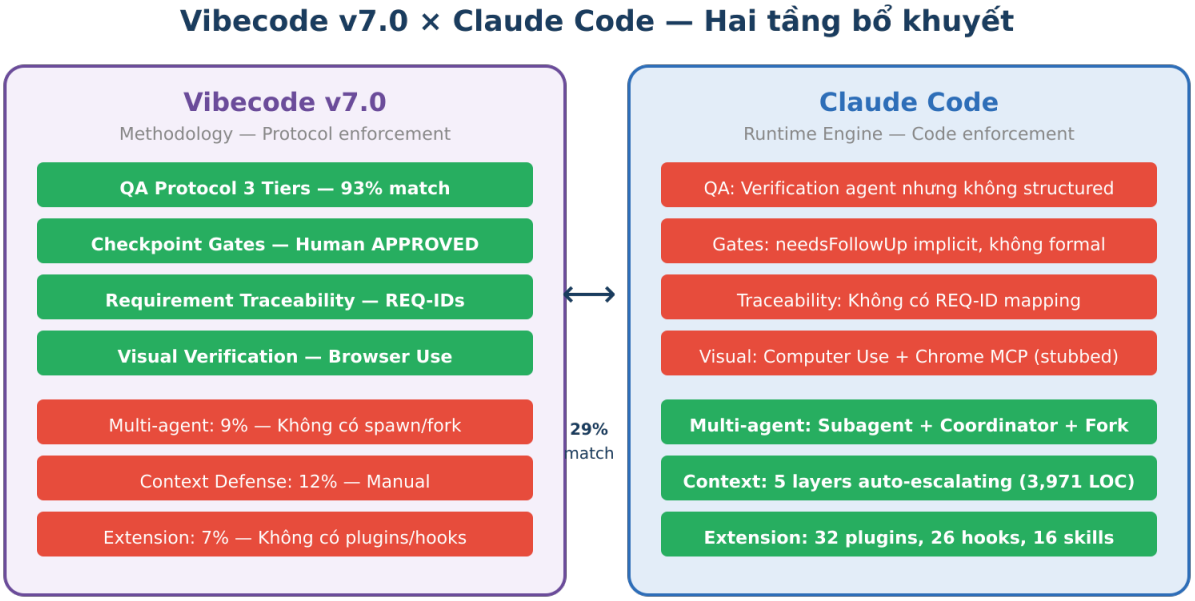
			fork)
Context và memory	3	**12%**	Claude Code (5 layers)
Quality assurance	4	**93%**	Vibecode (QA, tracing)
Extension ecosystem	3	**7%**	Claude Code (plugins, hooks)

Tổng: 29% matching = 71% gap. Nhưng gap không cân đối: Vibecode thiếu ở runtime (concurrency, multi-agent, context defense, hooks). Claude Code thiếu ở methodology (QA protocol, requirement tracing, checkpoint gates, visual verification). Mỗi bên mạnh đúng chỗ bên kia yếu.

* * *

Hình 13.1: Vibecode v7.0 × Claude Code

(Chương 13)



29% matching tổng thể. Vibecode dẫn QA (93%). Claude Code dẫn multi-agent, context defense, extension. Hai hệ thống bổ khuyết.

13.3 6 cơ chế v7.0 và nguồn gốc Claude Code

Vibecode v7.0 học 6 cơ chế từ Claude Code, dịch từ code enforcement sang methodology enforcement:

Cơ chế v7.0	Claude Code gốc	Vibecode dịch thành
Role enforcement	INTERNAL_WORKER_TOOLS restricted set (4 tools only)	FORBIDDEN action list. Violation = output invalid.
Multi-builder	Coordinator + N Workers. Coordinator cấm code.	Chủ thầu + Đội Thợ. Chủ thầu chỉ assign và review.
TIP concurrency	isConcurrencySafe() per tool, default false.	CONCURRENCY: SAFE|EXCLUSIVE per TIP, default EXCLUSIVE.
Context defense	5 layers: truncate → micro → auto → reactive → collapse	5 layers: carry → forward → milestone → handover → X-Ray
TIP → Skill	Skills: reusable prompts, 3 discovery modes, auto-activate.	TIP dùng 3x → promote thành Skill. Auto-activate by path.
Recovery	Retry ×3 → fallback model → circuit breaker.	Attempt 1-3 → Debug Protocol → Escalate → Re-design.

Khác biệt cốt lõi: Claude Code enforce bằng code (restricted tool list, feature gates). Vibecode enforce bằng protocol (FORBIDDEN list, Completion Report review, checkpoint gates). Cùng triết lý "constraint thay cho discipline," khác cơ chế. Cả hai cần: code enforcement cho runtime safety, protocol enforcement cho human oversight.

* * *

13.4 Những gì Vibecode giữ mà Claude Code không có

Bốn điểm Vibecode đạt 93-100% matching là những gì Claude Code thiếu và cần:

1. Checkpoint gates: 6 gates với checklist cụ thể giữa mỗi bước. Claude Code không có formal gates — nó dùng stop_reason + needsFollowUp, implicit. Vibecode gates là explicit: Human phải reply “APPROVED” để qua gate. Không có auto-approve.

2. QA Protocol 3 tiers: Tier 1 (mandatory core), Tier 2 (edge cases), Tier 3 (performance/security). Claude Code có Verification agent nhưng không có structured protocol để đảm bảo coverage. Vibecode RRI-T methodology (5 Personas × 7 Dimensions × 8 Stress Axes) là comprehensive framework mà agent-based verification không thể thay thế.

3. Requirement traceability: REQ-ID matrix từ RRI → Blueprint → TIP → VERIFY. Mỗi yêu cầu có ID, được trace qua toàn bộ lifecycle. Claude Code không có — không cách nào verify “mọi yêu cầu đã được implement.”

4. Visual verification: Browser Use integration (open → state → click → screenshot → eval). Claude Code có Computer Use và Chrome MCP nhưng stubbed trong leak. Vibecode có và đang dùng.

Đây là bản sắc Vibecode: con người chịu trách nhiệm, con người approve mọi gate, con người verify output. AI là builder xuất sắc nhưng không được tự phê duyệt sản phẩm của mình. Claude Code cho AI tự quyết stop_reason — phù hợp cho developer tool. Vibecode giữ Human approve mọi gate — phù hợp cho production system có trách nhiệm.

* * *

13.5 Kết luận chương

Vibecode v7.0 là kết quả của việc học từ Claude Code mà không mất bản sắc. 6 cơ chế mới từ runtime engine, 4 điểm mạnh giữ nguyên từ methodology. Matching từ 29% lên ước tính ~65% sau v7.0 upgrade, với 35% còn lại là những gì chỉ runtime code mới giải được (streaming execution, worktree isolation, real-time partitioning).

* * *

Chương tiếp theo sẽ áp dụng 18 patterns vào 5 hệ thống thực tế: ERP platform, multi-agent simulation, AI CRM, autonomous monitoring, và content processing pipeline.

CHƯƠNG 14

5 Blueprint ứng dụng

> *"Pattern không có giá trị cho đến khi được áp dụng vào hệ thống thực. Mỗi blueprint sau đây là một cách đọc 18 patterns qua lăng kính của một loại hệ thống cụ thể."*

14.1 Blueprint 1: ERP Platform với plugin architecture

Bài toán: Nền tảng ERP gồm nhiều app (MRP, HRM, CRM, PM...) chạy trên subdomains. Auth Gateway làm SSO. Cần: apps giao tiếp không hardcode, mở rộng không sửa core.

Patterns áp dụng: #14 (4-point extension) — mỗi app là plugin cung cấp agents + commands + hooks + servers. #15 (security sandbox) — app A không được tự gán quyền truy cập data app B. #16 (reconciliation install) — apps declared trong config, system reconcile với actual state. #10 (permission classification) — mỗi API call được classify theo semantics trước khi execute.

Kết quả: Integration cost O(n) thay vì O(n^2). App mới chỉ cần implement plugin interface, không cần biết apps khác tồn tại. Hook events thay thế direct API calls giữa apps.

* * *

14.2 Blueprint 2: Multi-agent simulation

Bài toán: Mô phỏng nhiều autonomous units hoạt động đồng thời trong cùng không gian. Mỗi unit có mission riêng, phải phối hợp mà không xông đột.

Patterns áp dụng: #7 (coordinator restriction) — coordinator chỉ điều phối, không tự thực thi mission. #4 (concurrency partition) — scan terrain (SAFE) chạy song song, thay đổi mission params (EXCLUSIVE) chạy tuần tự. #8 (fork isolation) — mỗi unit có workspace riêng, merge sau. #6 (context modifier) — unit scan vùng mới update shared map qua modifier chain. #5 (streaming execution) — units bắt đầu hành động ngay khi nhận mission segment.

Kết quả: Parallel execution tự nhiên từ concurrency classification. Không race condition nhờ workspace isolation. Coordinator pattern buộc delegation thay vì centralized execution.

* * *

14.3 Blueprint 3: AI-powered CRM

Bài toán: CRM với AI compliance matrix, RFP parsing, lead scoring. Sessions dài (phân tích documents lớn). Multi-user access.

Patterns áp dụng: #1 (async generator) — query loop cho compliance analysis, yield partial results cho UI. #9 (context defense) — 5 layers giữ session sống khi phân tích documents dài. #10 (permission classification) — classify CRM actions (read contact vs delete deal vs export data). #3 (escalating recovery) — API rate limit → retry → fallback model → queue.

Kết quả: AI compliance không crash khi document vượt context window. Permission classification ngăn AI tự ý xóa deals. Recovery đảm bảo continuity khi API đứt.

* * *

14.4 Blueprint 4: Autonomous monitoring system

Bài toán: Hệ thống giám sát chạy 24/7. Agents phát hiện anomaly, phân tích root cause, đề xuất fix. Human approve trước khi apply.

Patterns áp dụng: Background tasks (Chương 7) — monitor agents chạy nền, disk-based output, stall detection. #2 (stop-reason state machine) — agent tự quyết khi nào cần escalate vs tự xử lý. #3 (escalating recovery) — minor anomaly → auto-resolve, major → alert human, critical → page on-call. Memory system (Chương 8) — agents nhớ patterns từ incidents trước.

Kết quả: Monitoring không mỏi (agents chạy nền vô thời hạn). Graduated response (không mọi alert đều page human). Institutional memory (agents học từ incidents cũ).

* * *

14.5 Blueprint 5: Content processing pipeline

Bài toán: Xử lý documents lớn (dịch thuật, phân tích, tóm tắt) qua nhiều giai đoạn. Intermediate results cần review.

Patterns áp dụng: #1 (async generator) — pipeline yield intermediate results cho review. #5 (streaming execution) — bắt đầu stage 2 khi stage 1 output đầu tiên sẵn sàng. #11 (conditional skill activation) — skill dịch thuật tự activate khi touch .md files. #12 (shell-in-prompt) — inject file metadata (word count, language detect) vào prompt tự động.

Kết quả: Pipeline không block — human review intermediate results while pipeline continues. Skills tự load theo file type. Context injection tự động.

* * *

14.6 Pattern mapping tổng hợp

#	BP1 ERP	BP2 Sim	BP3 CRM	BP4 Monitor	BP5 Content
1			✓		✓
3			✓	✓	
4		✓			
5		✓			✓
7		✓			
8		✓			
9			✓	✓	
10	✓		✓		
14	✓				
15	✓				

Mỗi blueprint dùng 3-5 patterns. Không blueprint nào dùng tất cả 18. Patterns là menu, không phải checklist — chọn theo bài toán, không áp toàn bộ.

* * *

Chương tiếp theo sẽ trả lời câu hỏi triết học: trong một thế giới agents ngày càng tự chủ, vai trò của con người là gì?

CHƯƠNG 15

Con người trong vòng lặp

> *"Human-in-the-loop không phải khẩu hiệu. Nó là architectural decision. Quyết định này có giá — và giá đó đáng trả."*

15.1 Hai triết lý, không mâu thuẫn

Claude Code cho AI tự quyết stop_reason. Vòng lặp chỉ dừng khi LLM kết luận xong (needsFollowUp === false). Con người có thể abort bất kỳ lúc nào, nhưng default path là AI tự trị.

Vibecode giữ con người approve mọi gate. Không TIP nào được thực thi mà không qua checkpoint. Không sản phẩm nào được ship mà không qua VERIFY + Human "SHIP" decision. Default path là human oversight.

Hai triết lý này không mâu thuẫn. Chúng phục vụ bối cảnh khác nhau.

* * *

15.2 Khi nào cho agent tự trị

Bối cảnh phù hợp cho autonomy:

Personal tool: developer dùng Claude Code trên laptop cá nhân. Sai thì revert. Risk thấp. Latency quan trọng hơn safety. Agent tự trị tăng tốc độ 10x.

Reversible actions: mọi hành động đều undo được. Git revert, database rollback, file restore. Sai không mất gì vĩnh viễn. Trust agent tạo value; revert khi sai.

Low-stakes: prototype, experiment, personal project. Failure cost thấp. Learning cost cao nếu phải approve từng bước.

* * *

15.3 Khi nào giữ human-in-the-loop

Bối cảnh bắt buộc human oversight:

Production system: code chạy phục vụ khách hàng. Sai = downtime = mất tiền = mất uy tín. Không thể "revert rồi thử lại." Human review trước khi deploy.

Irreversible actions: xóa production database, gửi email cho 10.000 khách hàng, ký hợp đồng. Không revert được. Phải đúng lần đầu.

Compliance/Legal: sản phẩm phải đáp ứng regulations (NDAA, GDPR, SOC2). AI không chịu trách nhiệm pháp lý. Con người chịu. Con người phải verify.

Multi-stakeholder: nhiều team, nhiều owner. AI không biết political context, priority trade-offs, interpersonal dynamics. Con người navigate.

* * *

15.4 Autonomy dial — Spectrum, không binary

Thay vì binary "tự trị hay không," thiết kế "mức tự trị điều chỉnh được per-context." Claude Code đã làm điều này với 7 permission modes (default → plan → acceptEdits → auto → dontAsk → bypass). Mỗi mode là một điểm trên spectrum từ "hỏi mọi thứ" đến "cho phép mọi thứ."

Vibecode v7.0 có thể áp dụng tương tự: CONCURRENCY tag (SAFE/EXCLUSIVE) là autonomy dial ở task level. SAFE TIPs cho phép Thợ chạy song song không cần approve từng cái. EXCLUSIVE TIPs yêu cầu review tuần tự. Mức autonomy tùy theo risk của task, không phải tùy theo preference của user.

* * *

15.5 Trách nhiệm không uủy quyền được

AI có thể viết code tốt hơn nhiều developer. AI có thể tìm bug nhanh hơn. AI có thể refactor clean hơn. Nhưng AI không chịu trách nhiệm khi sản phẩm gây hại.

Khi production system crash lúc 2 giờ sáng, không ai gọi điện cho Claude. Khi dữ liệu khách hàng bị lộ, không ai kiện Claude. Khi sản phẩm không đạt compliance, không ai phạt Claude. Con người chịu trách nhiệm. Vì vậy con người phải có quyền kiểm soát.

Đây là lý do Vibecode tồn tại: không phải vì AI không đủ giỏi. Mà vì trách nhiệm không uủy quyền được. Người ký tên trên sản phẩm phải có quyền duyệt mọi gate. Checkpoint gates không phải overhead — chúng là cơ chế cho người chịu trách nhiệm thực thi trách nhiệm.

* * *

15.6 Tương lai: con người làm gì khi AI làm hết?

Câu trả lời: con người làm những gì AI không thể chịu trách nhiệm.

Ra quyết định chiến lược: AI đề xuất 3 hướng kiến trúc. Con người chọn dựa trên business context, team capability, và long-term vision mà AI không thấy.

Chịu trách nhiệm với stakeholders: Khi khách hàng hỏi tại sao feature chưa xong, con người trả lời. Khi CEO hỏi budget, con người giải trình. AI làm nhưng con người own.

Navigate ambiguity: "Requirement này mâu thuẫn với requirement kia." AI escalate. Con người quyết định: ưu tiên cái nào, đánh đổi gì.

Verify và ship: VERIFY step trong Vibecode không phải kiểm tra AI làm đúng chưa (AI có thể tự test). Nó kiểm tra sản phẩm có phù hợp với những gì con người cam kết với stakeholders không. Chỉ con người mới biết.

* * *

Chương cuối cùng sẽ nhìn về phía trước: 18 patterns này không chỉ cho coding tool. Chúng cho bất kỳ agentic system nào.

CHƯƠNG 16

Tương lai: Agentic OS cho mọi domain

> *"18 patterns này được rút từ một coding tool. Nhưng chúng không chỉ về code. Chúng về cách điều phối AI agents — bất kể agents đó làm gì."*

16.1 Từ coding tool đến agentic operating system

Claude Code là coding tool. Nhưng kiến trúc của nó là agentic operating system. Query loop không biết nó đang chạy coding tasks — nó chỉ biết: gọi LLM, thực thi tools, lặp lại. Tool orchestration không biết tools làm gì — nó chỉ biết: cái nào safe, cái nào exclusive. Context defense không biết conversation về gì — nó chỉ biết: tokens còn bao nhiêu.

Abstractions này domain-agnostic. Thay BashTool bằng RobotControlTool, hệ thống vẫn hoạt động. Thay FileReadTool bằng SensorDataTool, query loop vẫn lặp. Thay codebase context bằng patient medical history, context defense vẫn compact.

* * *

16.2 5 domains chờ đợi

Robotics và autonomous systems

Query loop cho autonomous decision making. Concurrency partitioning cho multi-sensor fusion (camera read SAFE, motor control EXCLUSIVE). Fork isolation cho multi-robot coordination. Permission classification cho safety-critical actions.

Healthcare AI assistants

Context defense cho long patient histories (hàng năm dữ liệu). Permission classification cho medical actions (read chart SAFE, prescribe EXCLUSIVE, require doctor approval). Memory system cho treatment continuity qua nhiều sessions.

Legal và compliance

Requirement traceability (Vibecode REQ-IDs) cho regulation mapping. Skill system cho jurisdiction-specific legal templates. Checkpoint gates cho multi-party review workflows.

Education và tutoring

Subagent pattern cho specialized tutors (math agent, writing agent, science agent). Memory system cho student progress tracking. Conditional skills auto-activate by subject area.

Enterprise workflow automation

Plugin architecture cho department-specific extensions. Hook events cho cross-department coordination. Coordinator pattern cho process orchestration. Background tasks cho long-running approvals.

* * *

16.3 Những gì chưa có

18 patterns giải quyết nhiều bài toán. Nhưng còn những bài toán chưa ai giải:

Multi-model orchestration: Claude Code dùng một model chính + fallback. Tương lai cần: chọn model tối ưu per-task (fast model cho simple tasks, large model cho complex). Routing intelligence.

Cost optimization: 513K dòng code không có cost-aware scheduling. Tương lai cần: budget constraints, cost-per-task estimation, automatic downgrade khi budget cạn.

Formal verification: Vibecode có QA protocol nhưng manual. Tương lai cần: agent tự generate và chạy formal proofs, mathematical guarantees thay vì empirical testing.

Real-time collaboration: Claude Code là single-user tool. Tương lai cần: nhiều users + nhiều agents cùng workspace, real-time conflict resolution, shared context.

* * *

16.4 Lời kết

Ngày 31 tháng 3 năm 2026, một source map trong npm package vô tình lộ 513.000 dòng code. Từ đó, 18 architectural patterns được extracted, verified từng dòng, và đúc kết thành cuốn sách này.

Những patterns này không thuộc về Anthropic. Chúng thuộc về engineering. Async generator là pattern của JavaScript. Concurrency partitioning là pattern của systems programming. Escalating recovery là pattern của resilient systems. Defense-in-depth là pattern của security engineering.

Điều đặc biệt là chúng được kết hợp thành một hệ thống nhất quán, được validate bởi hàng triệu users, và bây giờ có thể học từ bởi bất kỳ ai đọc cuốn sách này.

Bắt đầu từ pattern nào? Bắt đầu từ Pattern #1 (async generator query loop). Nó là nền tảng. Mọi pattern khác đều gắn vào vòng lặp này.

* * *

> *"513.000 dòng code không tự nói. Nhưng nếu bạn biết đặt câu hỏi đúng, chúng trả lời mọi thứ."*

Phụ lục A: Bảng tra 18 Patterns

#	Pattern	Source location	Ch.
1	**Async generator as control flow**	query.ts:219-241	3
2	**Stop-reason state machine (needsFollowUp)**	query.ts:557-559	3
3	**Escalating recovery**	query.ts:1188-1242	3
4	**Concurrency-safe partitioning**	toolOrchestration.ts:91-120	4
5	**Streaming tool execution**	StreamingToolExecutor.ts (530 LOC)	4
6	**Context modifier chain**	Tool.ts:329, StreamingToolExecutor.ts:385	4
7	**Coordinator restriction (4 tools only)**	coordinatorMode.ts:29-34	6
8	**Fork isolation via Git worktree**	forkSubagent.ts:60-71	6
9	**5-layer context defense**	services/compact/ (3,971 LOC)	8

10	**Permission classification pipeline**	utils/permissions/ (9,409 LOC)	5
11	**Conditional skill activation by file path**	skills/ (4,066 LOC)	9
12	**Shell-in-prompt (dynamic context)**	skills/executeShell.ts	9
13	**Dynamic skill discovery (directory walk)**	skills/discoverSkillDirs.ts	9
14	**4-point plugin extension**	plugins/ (31,484 LOC)	10
15	**Security sandbox for plugins**	plugins/loadPluginAgents.ts	10
16	**Reconciliation-based install**	PluginInstallationManager.ts	10
17	**Pure-TS native module replacement**	native-ts/yoga-layout/ (2,578 LOC)	11
18	**Terminal-as-browser rendering**	ink/ (19,865 LOC)	11

Phụ lục B: Thuật ngữ

Thuật ngữ	Định nghĩa
Agent	Chương trình AI tự lặp, tự gọi tool, tự kiểm tra cho đến khi hoàn thành task
Agentic OS	Hệ điều hành cho AI agents: quản lý tài nguyên, phân quyền, điều phối
Compact / Compaction	Nén conversation để tiết kiệm context window
Compact boundary	Message đánh dấu ranh giới: mọi messages trước boundary bị loại
Concurrency-safe	Tool có thể chạy song song an toàn (không side-effect)
Context modifier	Callback thay đổi shared context sau tool execution
Context window	Giới hạn tokens LLM nhìn thấy trong một request
Coordinator	Agent chỉ điều phối, bị cấm dùng execution tools
Fork agent	Agent clone context + Git worktree riêng, merge sau
Human-in-the-loop	Con người có quyền can thiệp bất kỳ lúc nào
MCP	Model Context Protocol — protocol kết nối external tool servers
needsFollowUp	Boolean flag quyết định query loop tiếp tục hay dừng
Permission fatigue	Hiện tượng user approve mọi thứ vì mệt mỏi đọc
Query loop	Vòng lặp chính: LLM → tools → results → LLM → ...
Skill	Prompt template tái sử dụng, auto-activate theo ngữ cảnh
Subagent	Agent con, spawn bởi parent, có own query loop
TIP	Task Instruction Pack — đơn vị công việc trong Vibecode
Tool	Hành động cụ thể agent thực hiện (đọc file, chạy bash...)
Vibecode	Phương pháp luận AI-assisted development, 3 vai trò, 8 bước
Worker	Agent thực thi, có full tool access, không có quyền thiết kế
YOLO classifier	LLM classifier trong auto mode (tên nội bộ Anthropic)

Phụ lục C: Tham chiếu mã nguồn

Bảng tham chiếu các file chính trong Claude Code source (513K LOC):

File / Directory	LOC	Vai trò
src/query.ts	1,729	Query loop chính (while true)
src/services/tools/toolOrchestration.ts	~200	Partition + run tools
src/services/tools/StreamingToolExecutor.ts	530	Streaming tool execution
src/services/tools/toolExecution.ts	1,745	Core tool execution logic
src/services/compact/ (13 files)	3,971	Context defense 5 layers
src/utils/permissions/ (24 files)	9,409	Permission classification
src/tools/ (43 directories)	~50K	All tool implementations
src/tools/AgentTool/ (20 files)	6,782	Multi-agent spawning
src/coordinator/	~2K	Coordinator mode
src/tasks/ (12 files)	3,286	Background task system
src/memdir/ (8 files)	~2K	Memory system
src/ink/ (custom fork)	19,865	Terminal React renderer
src/native-ts/yoga-layout/	2,578	Flexbox engine (TS port)
src/skills/ (23 files)	4,066	Skill system
src/plugins/ (65+ files)	31,484	Plugin marketplace
src/Tool.ts	~800	Tool interface + defaults

Nguồn: fazxes/Claude-code (GitHub, March 31 2026). 486 extracted files, 15 git commits, 1,907 .ts/.tsx files, 512,996 LOC.

Phụ lục D: Thống kê sách

Metric	Giá trị
Số chương	**16**
Số phần	**5 (Nền tảng, Engine, Swarm, Ecosystem, Synthesis)**
Tổng paragraphs	**1.454**
Patterns extracted	**18 (10/10 verified)**
Blueprints ứng dụng	**5**
Nguồn code phân tích	**512.996 LOC TypeScript**
Phương pháp luận	**Vibecode v6.0 → v7.0**
Ngôn ngữ	**Tiếng Việt (bản gốc)**
Methodology	**Vibecode Kit v6.0 — 19 TIPs**